Clemson Means
BUSINESS

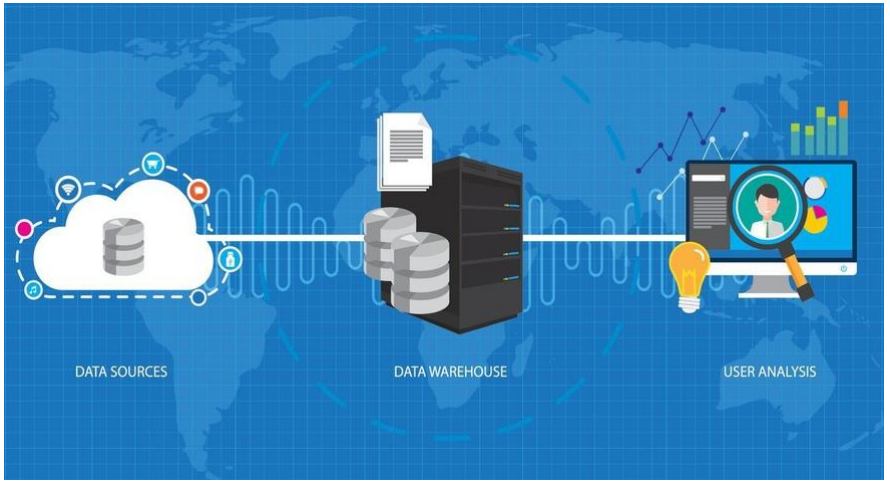
Chapter 5
Data Warehousing and Integration

Instructor: He Li

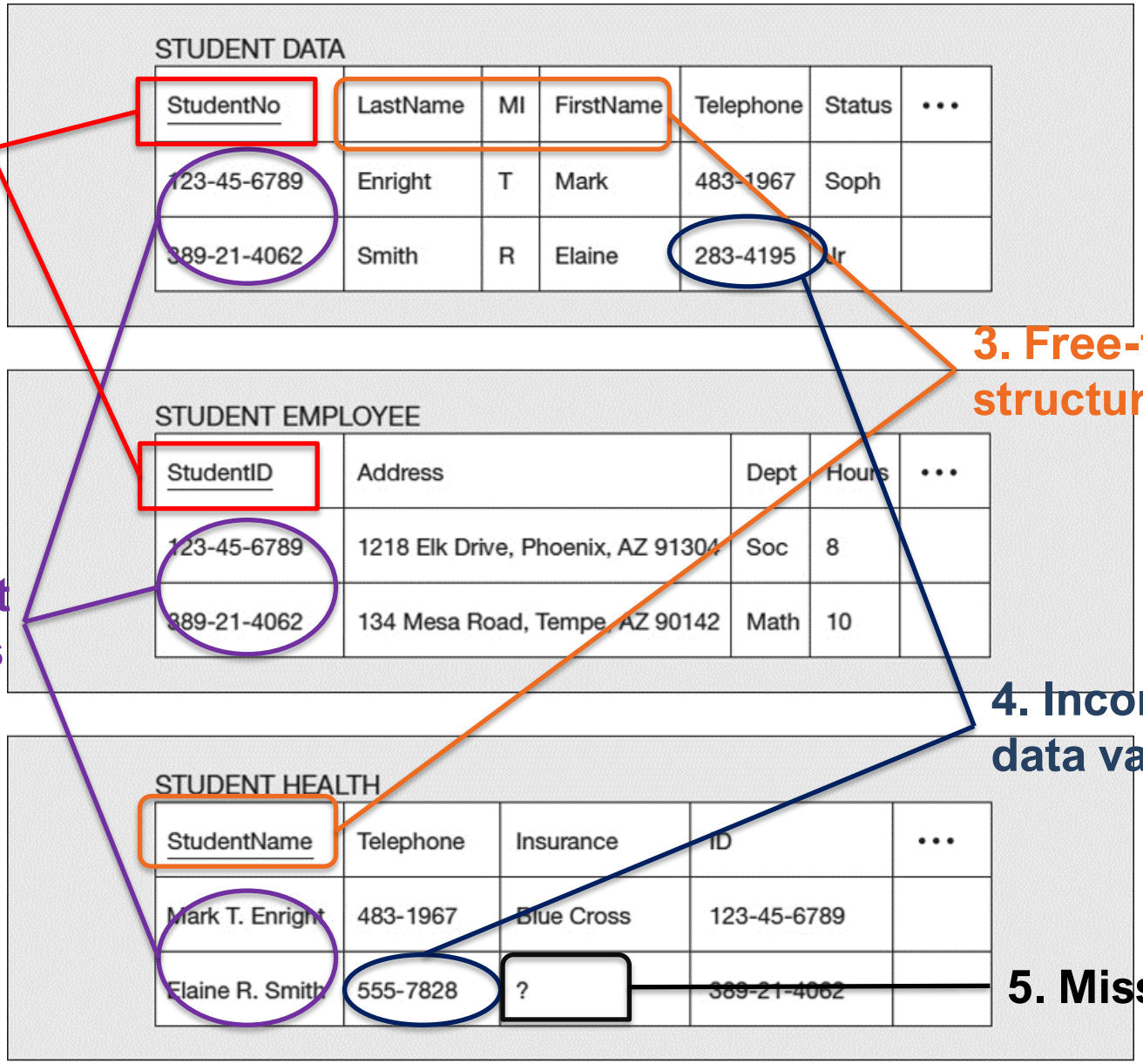
Data Warehouse

- **Definition:** A *subject-oriented*, *integrated*, *time-variant*, *non-updatable* collection of data used in support of management decision-making processes

- **Why Data Warehousing?**
  - Integrated, company-wide view of high-quality information (from disparate databases)
  - Separation of **operational** and **informational** systems and data (for improved performance)



Difficulty of Deriving Company-Wide View



2. Synonym

1. Inconsistent key structures

3. Free-form vs structured fields

4. Inconsistent data values

5. Missing data

Organizational Trends Motivating Data Warehouses

Department of MANAGEMENT Clemson University

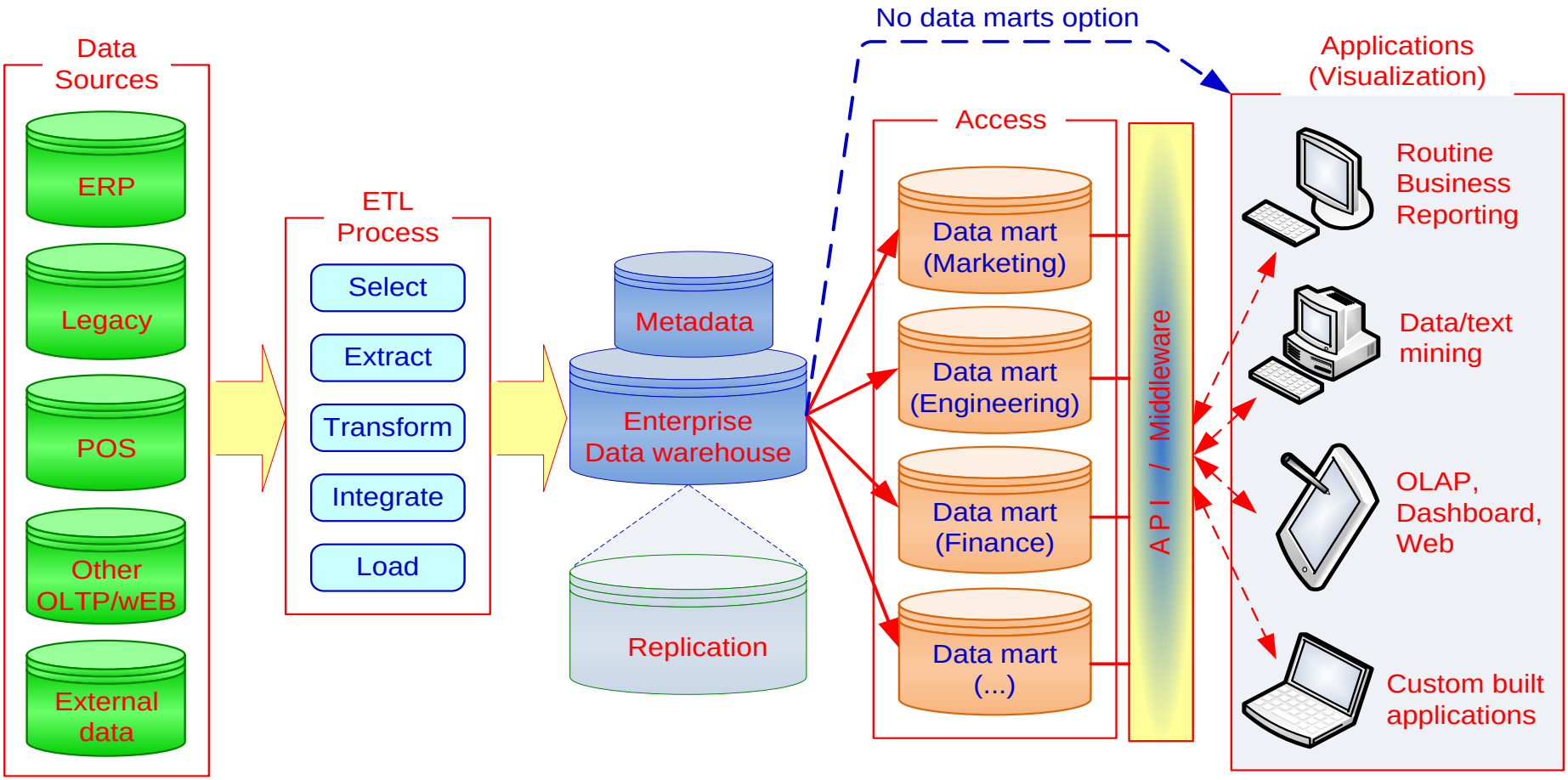
- No single system of records
- Multiple systems not synchronized
- Organizational need to analyze activities in a balanced way
- Customer relationship management
- Supplier relationship management

Operational System – a system that is used to run a business in real time, based on current data; also called a system of record

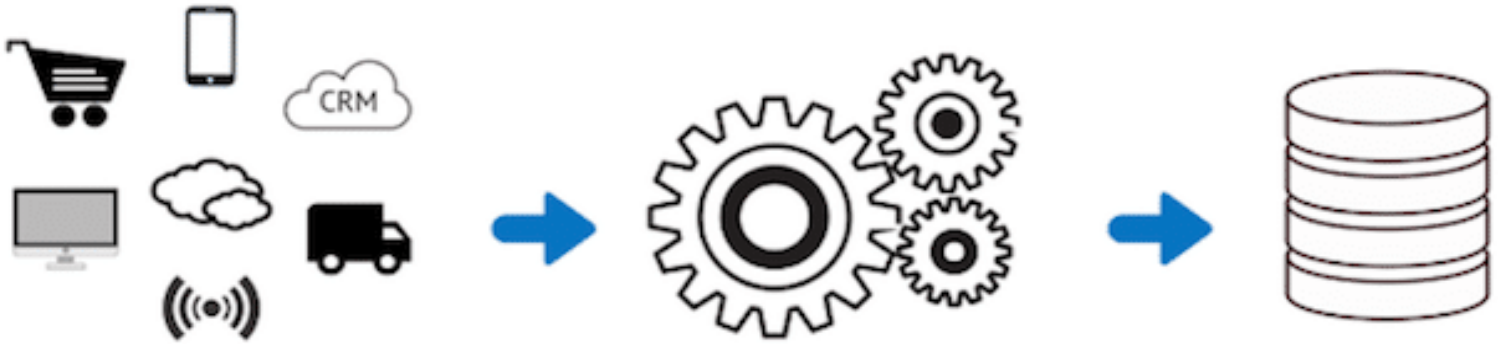
Informational System – a system designed to support decision making based on historical point-in-time and prediction data for complex queries or data-mining applications

Characteristic	Operational Systems	Informational Systems
Primary purpose	Run the business on a current basis	Support managerial decision making
Type of data	Current representation of state of the business	Historical point-in-time (snapshots) and predictions
Primary users	Clerks, salespersons, administrators	Managers, business analysts, customers
Scope of usage	Narrow, planned, and simple updates and queries	Broad, ad hoc, complex queries and analysis
Design goal	Performance: throughput, availability	Ease of flexible access and use
Volume	Many constant updates and queries on one or a few table rows	Periodic batch updates and queries requiring many or all rows

A Generic Data Warehousing Framework



ETL – Data Integration for Data Warehousing



reading data from one or more sources (e.g. OLTP databases, personal databases, spreadsheets)

converting the extracted data into the appropriate form, cleaning data

putting the data into the data warehouse

Data Integration

creating a unified view of business data

Department of MANAGEMENT Clemson University

Method	Pros	Cons
Consolidation (ETL)	• Users are isolated from conflicting workloads on source systems, especially updates. • It is possible to retain history, not just current values. • A data store designed for specific requirements can be accessed quickly. • It works well when the scope of data needs are anticipated in advance. • Data transformations can be batched for greater efficiency.	• Network, storage, and data maintenance costs can be high. • Performance can degrade when the data warehouse becomes quite large (with some technologies).
Federation (EII)	• Data are always current (like relational views) when requested. • It is simple for the calling application. • It works well for read-only applications because only requested data need to be retrieved. • It is ideal when copies of source data are not allowed. • Dynamic ETL is possible when one cannot anticipate data integration needs in advance or when there is a one-time need.	• Heavy workloads are possible for each request due to performing all integration tasks for each request. • Write access to data sources may not be supported.
Propagation (EAI & EDR)	• Data are available in near real time. • It is possible to work with ETL for real-time data warehousing. • Transparent access is available to the data source.	• There is considerable (but background) overhead associated with synchronizing duplicate data.

Data Warehouse v.s. Data Mart

Department of MANAGEMENT Clemson University

Data Warehouse	Data Mart
Scope	**Scope**
• Application independent • Centralized, possibly enterprise-wide • Planned	• Specific DSS application • Decentralized by user area • Organic, possibly not planned
Data	**Data**
• Historical, detailed, and summarized • Lightly denormalized	• Some history, detailed, and summarized • Highly denormalized
Subjects	**Subjects**
• Multiple subjects	• One central subject of concern to users
Sources	**Sources**
• Many internal and external sources	• Few internal and external sources
Other Characteristics	**Other Characteristics**
• Flexible • Data oriented • Long life • Large • Single complex structure	• Restrictive • Project oriented • Short life • Start small, becomes large • Multi, semi-complex structures, together complex

Data Warehouse Development

Department of MANAGEMENT Clemson University

Inmon Model: EDW Approach (Top-Down)

- Provides a highly consistent dimensional view of data across data marts as all data marts are loaded from the centralized repository (Data Warehouse).
- Flexible to support business changes as it looks at the organization as whole.
- Generating a new data mart against the data stored in the data warehouse relatively simple.
- Large-scale and scope of project
- Up-front cost
- Long duration; may be inflexible and unresponsive to changing business needs during implementation

Kimball Model: Data Mart Approach (Bottom-Up)

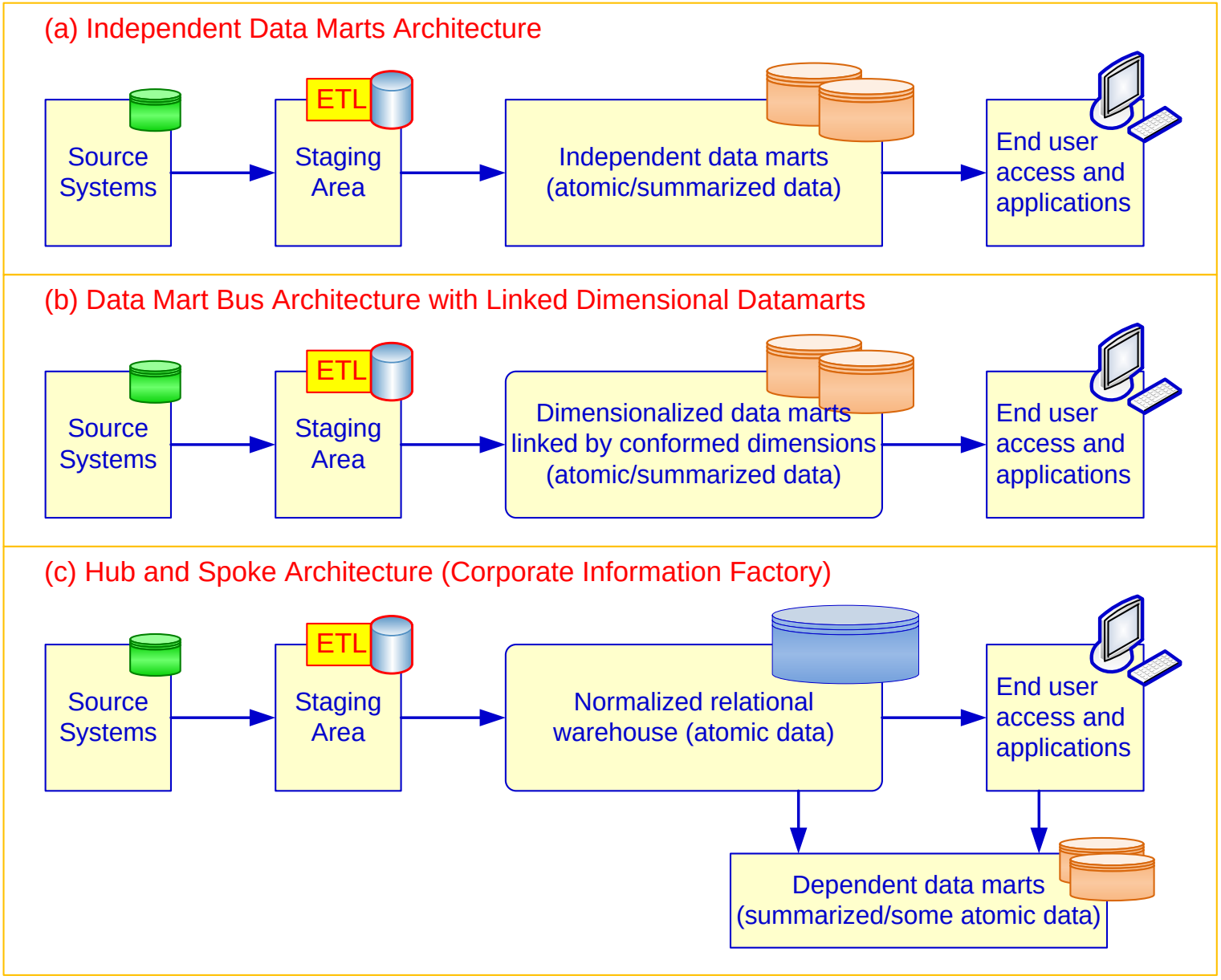
- Emphasizes the value of the data warehouse to business users as quickly as possible.
- Focuses on each individual business process -> quick return on investment
- May lack the big picture of the enterprise data warehouse by missing some dimensions or by creating redundant dimensions
- Lower cost
- Fairly simple

Choosing Data Warehouse Development Approach

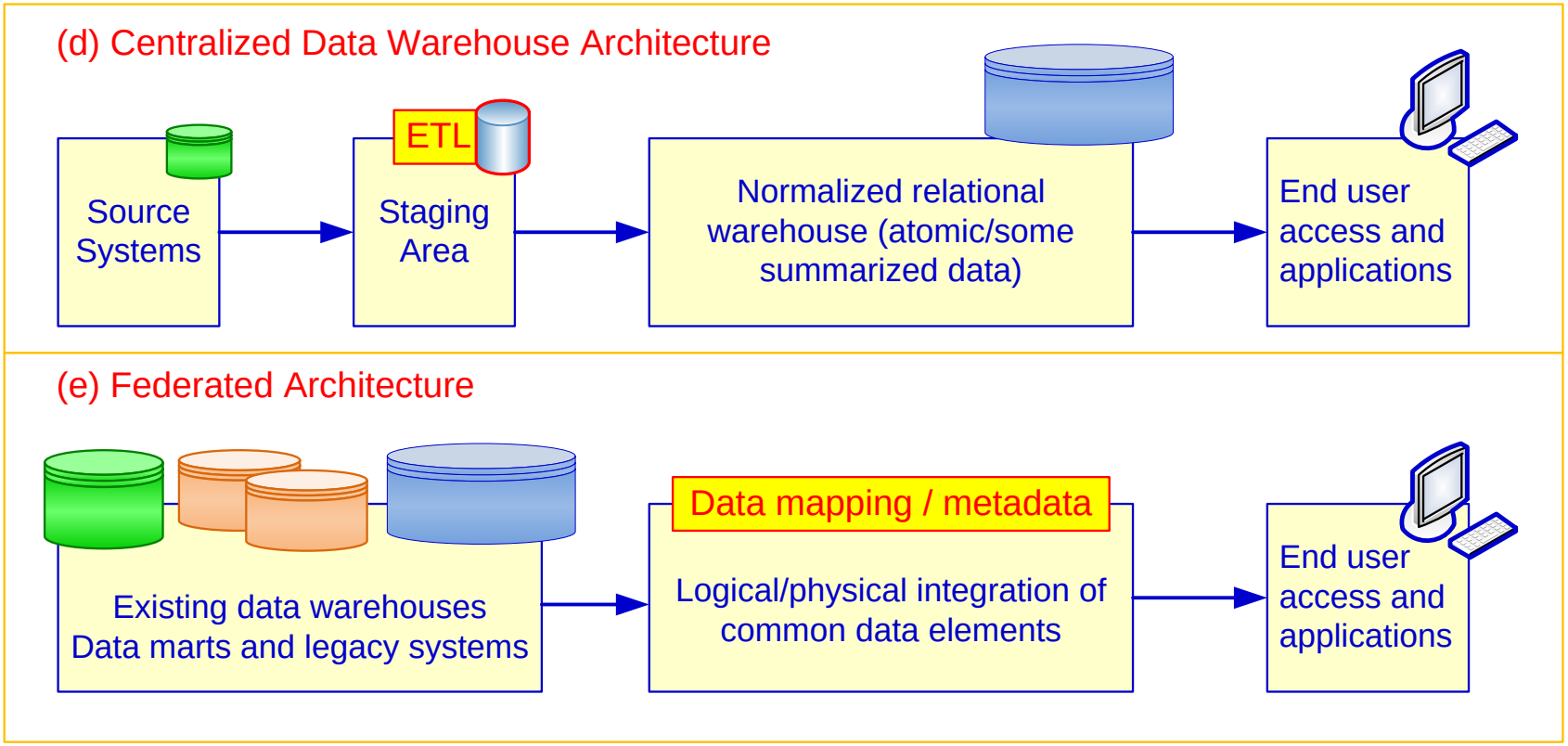
Department of MANAGEMENT Clemson University

Characteristic	Favors Kimball	Favors Inmon
Persistency of data	Source systems are relatively stable	High rate of change from source systems
Staffing and skills requirements	Small teams of generalists	Larger team(s) of specialists
Time to delivery	Need for the first data warehouse application is urgent	Organization's requirements allow for longer start-up time
Cost to deploy	Lower start-up costs, with each subsequent project costing about the same	Higher start-up costs, with lower subsequent project development costs

Alternative DW Architectures

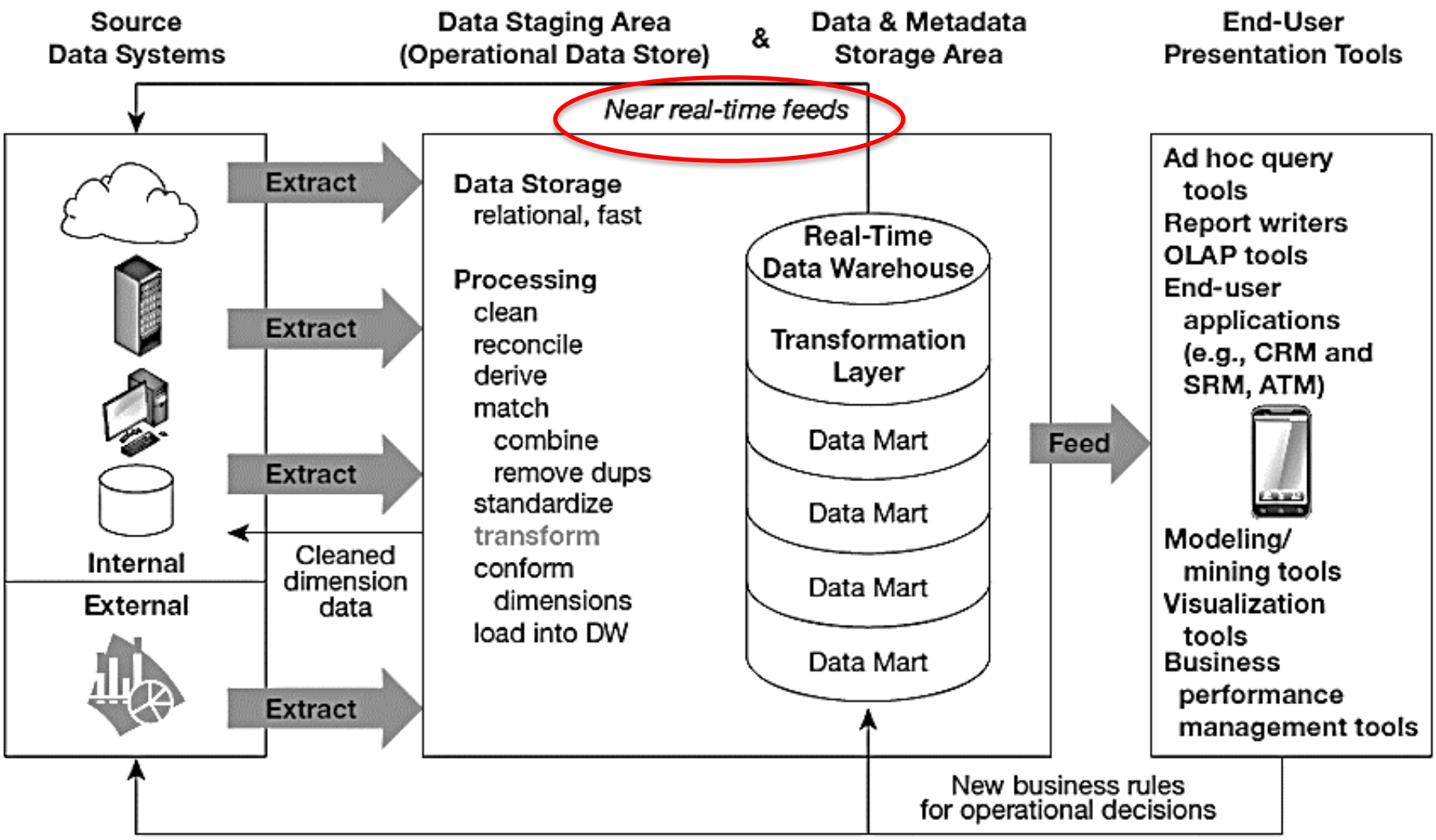


Alternative DW Architectures (cont.)



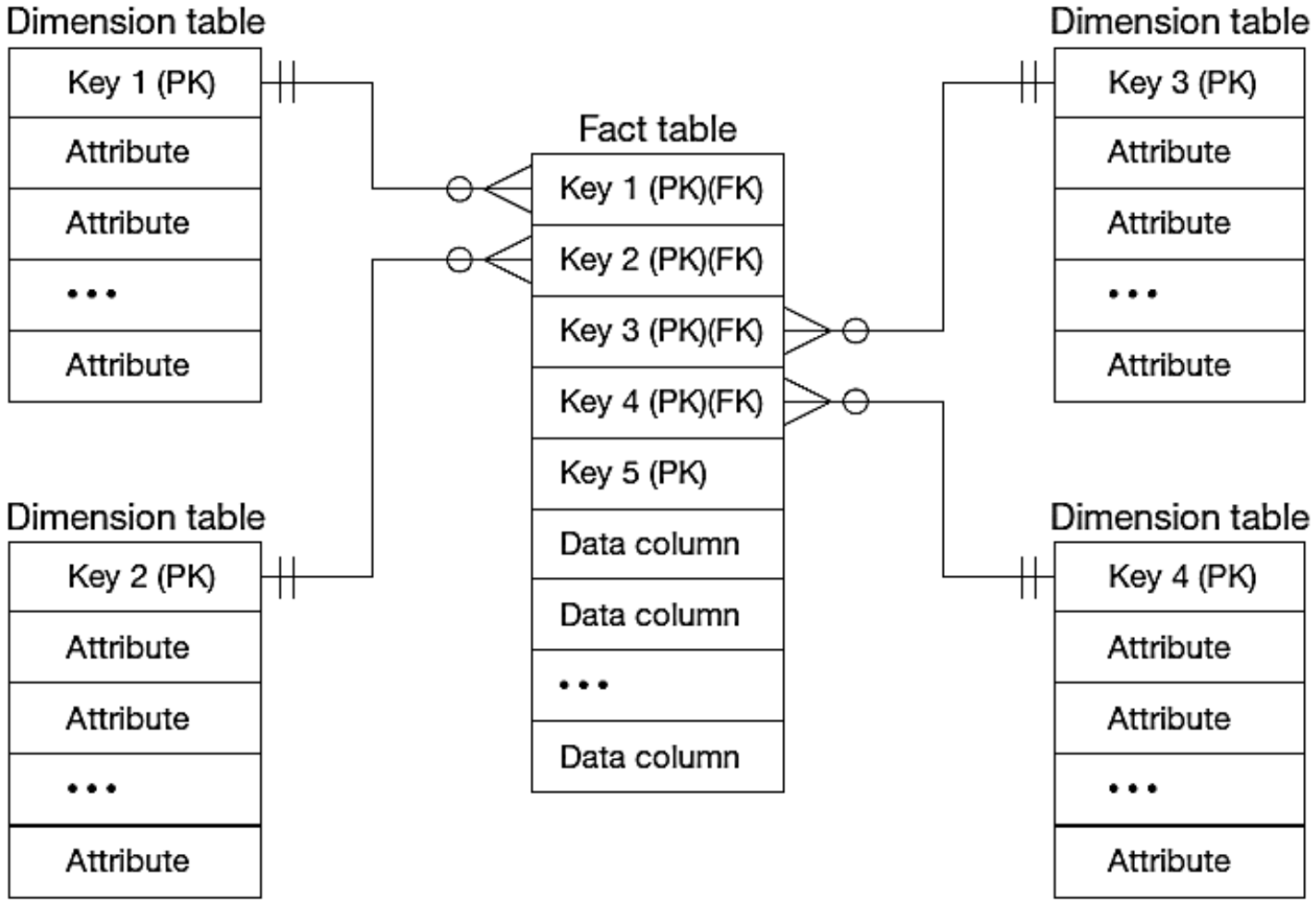
- Each architecture has advantages and disadvantages!

Logical Data Mart and Real-time Data Warehouse Architecture



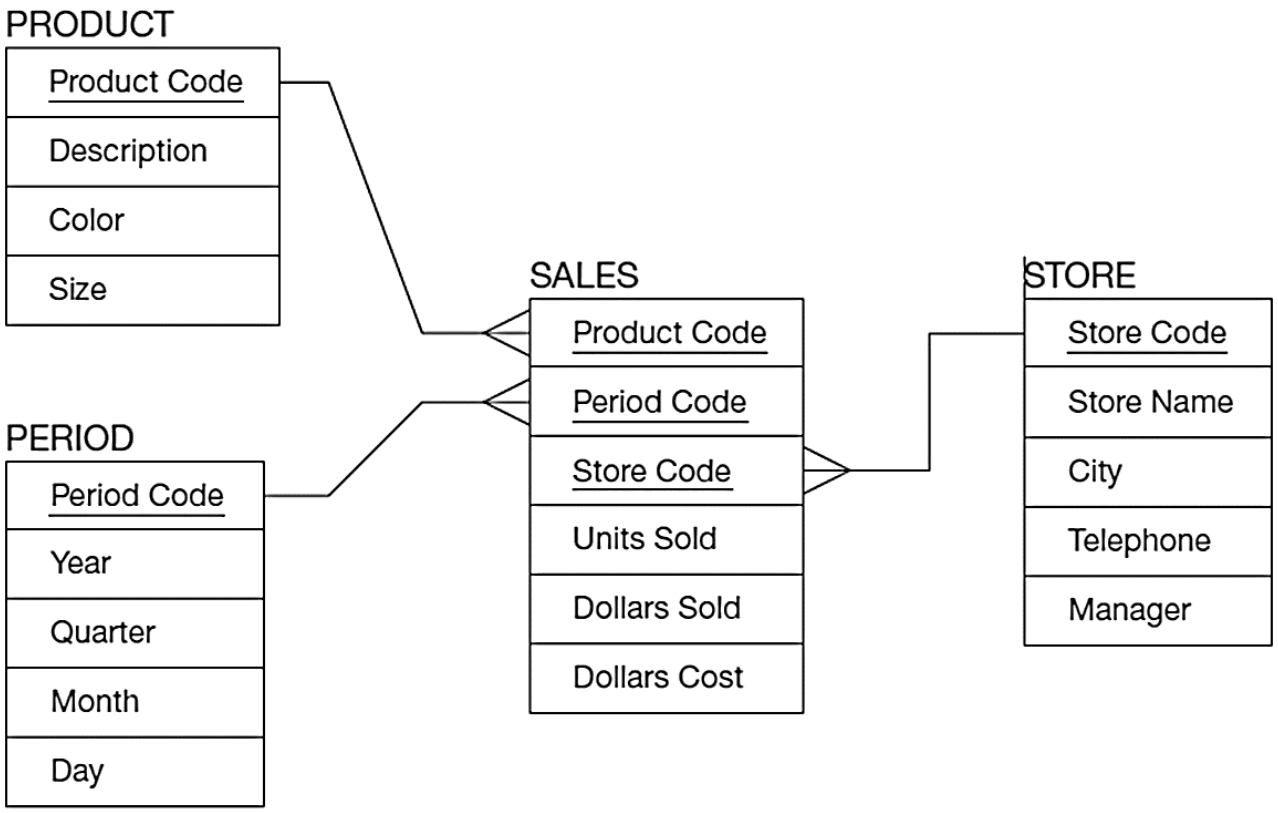
Dimensional Modeling: Star Schema

- A simple database design in which dimensional data are separated from fact or event data.

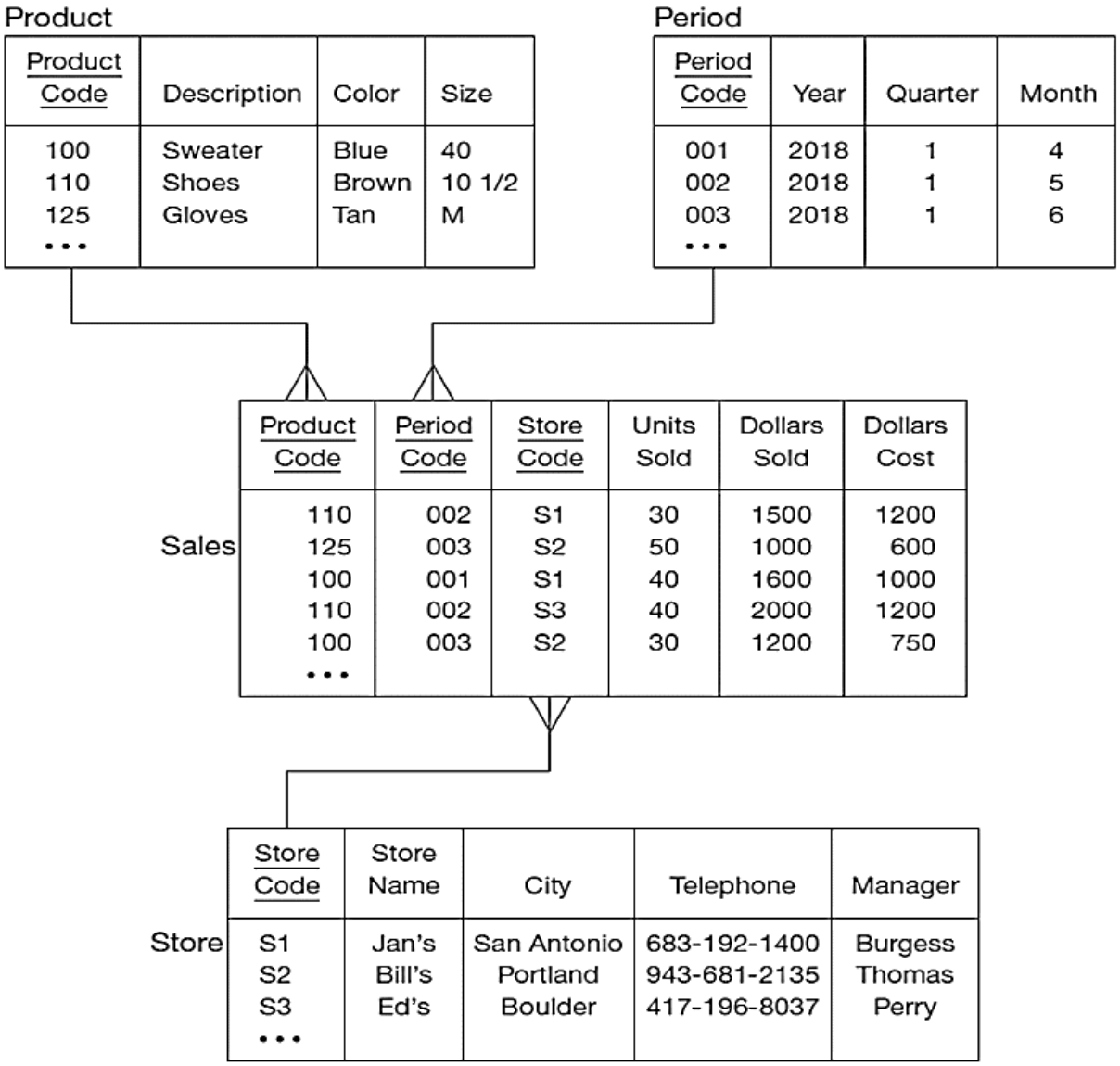


Example: Star Schema

- Fact table provides statistics for sales broken down by product, period, and store dimensions



Example: Star Schema



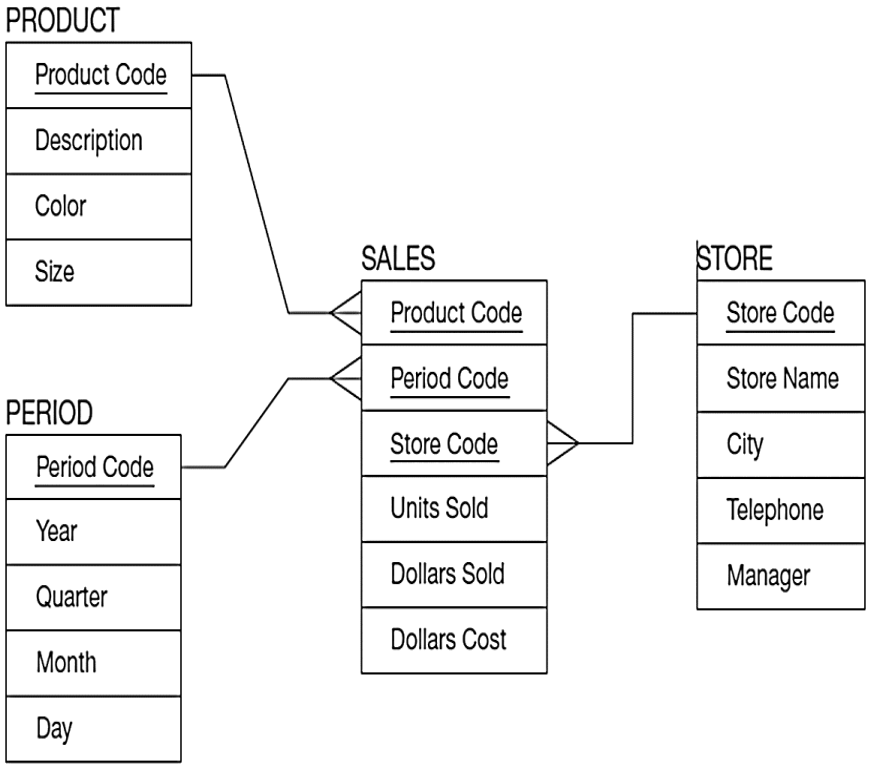
Specific Issues

- **Surrogate Key:** dimension table keys should be surrogate (non-intelligent and non-business related);
- **Granularity of Fact Table:** What level of detail do you want?
  - Transactional grain – finest level
  - Aggregated grain – more summarized
  - Finer grains brings better market basket analysis capability
  - Finer grain implies more dimension tables, more rows in fact table
  - In Web-based commerce, finest granularity is a click
- **Duration of the Database**
  - Natural duration – 13 months or 5 quarters
  - Financial institutions may need longer duration
  - Older data is more difficult to source and cleanse

Specific Issues – size of fact table

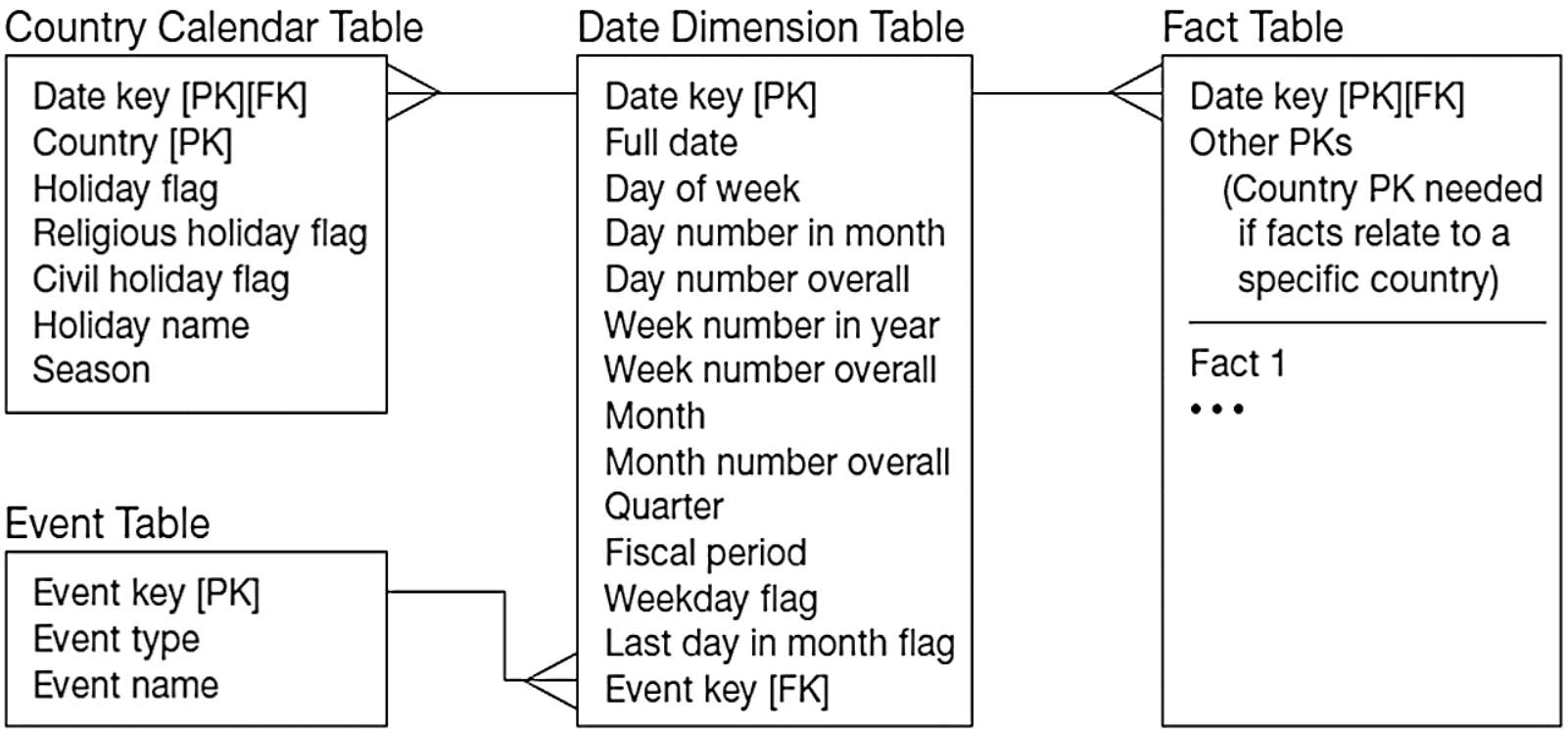
- Depends on number of dimensions and grain of the fact table
- Number of rows = product of number of possible values for each dimension associated with the fact table

- Example: Assume the following:
  - Total number of stores = 1,000
  - Total number of products = 5,000
  - Total number of periods = 24 (two years' worth of monthly data)
- Total rows calculated as follows:
  - Total rows = 1,000 stores × 5,000 active products × 24 months = 120,000,000 rows



Specific Issues – modeling dates

- Fact tables contain time-period data, so date dimensions are important.

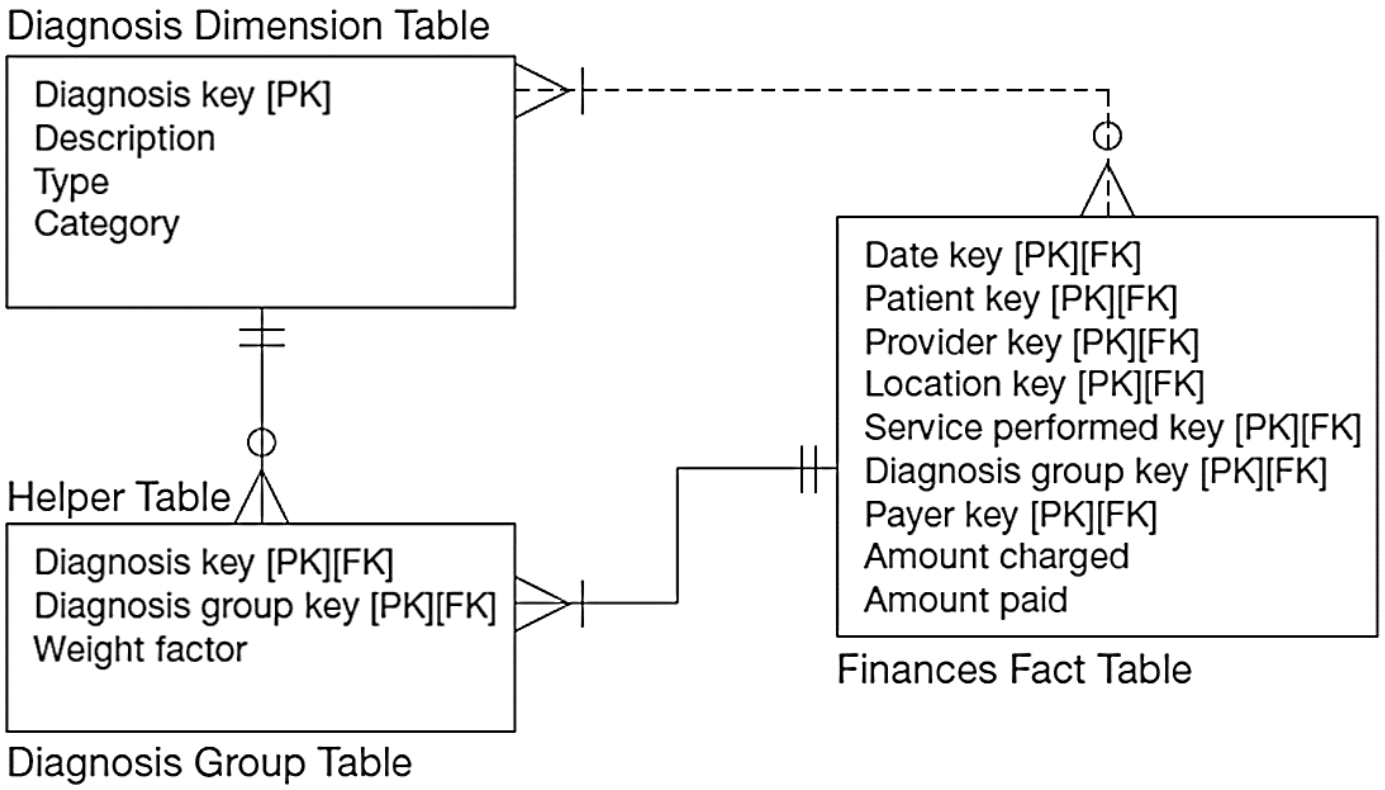


Snowflake Schema – Normalizing Dimension Tables

- Multivalued Dimensions
  - Facts qualified by a set of values for the same business subject
  - Normalization involves creating a table for an associative entity between dimensions
- Hierarchies
  - Sometimes a dimension forms a natural, fixed-depth hierarchy
  - Design options
  - Include all information for each level in a single denormalized table
  - Normalize the dimension into a nested set of 1:N table relationships

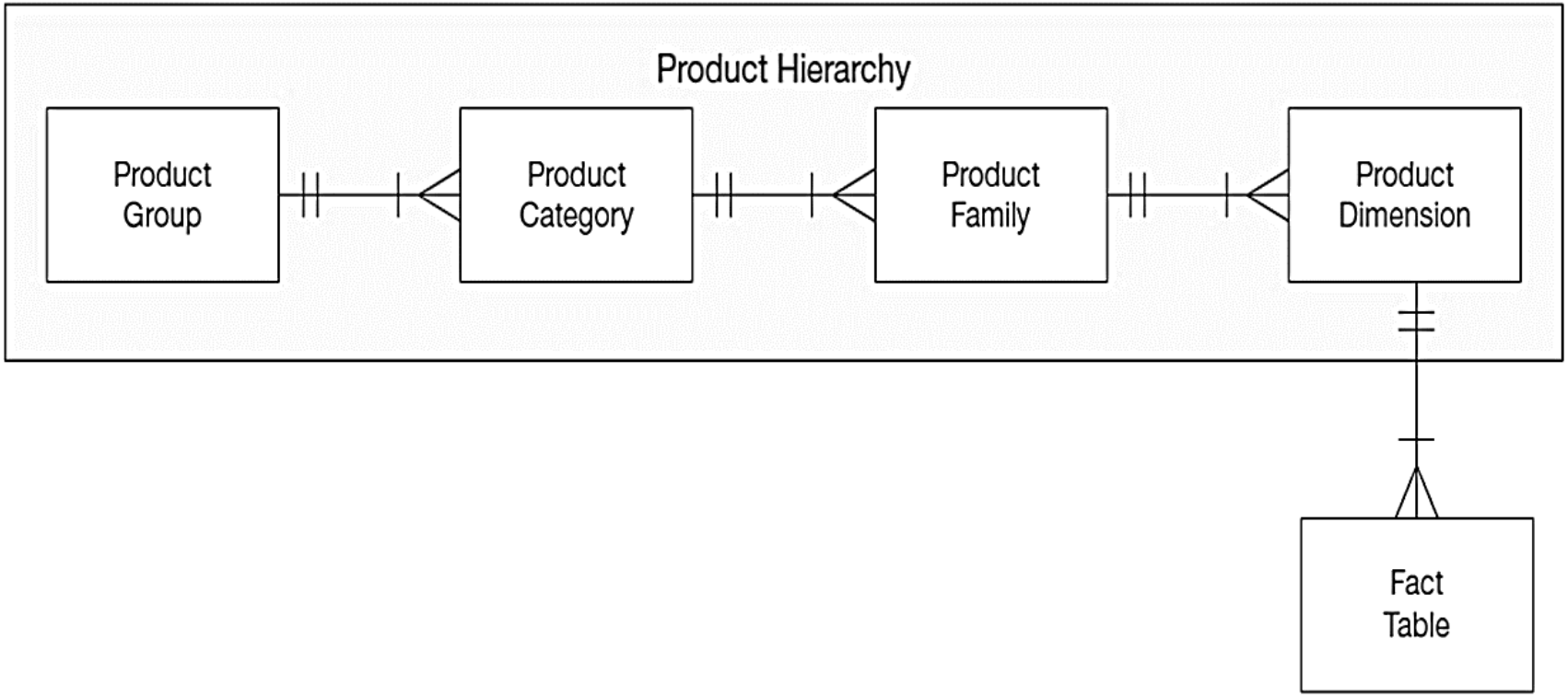
Example: Normalizing Multivalued Dimensional Tables

- A helper table is an associative entity that implements a M:N relationship between dimension and fact.



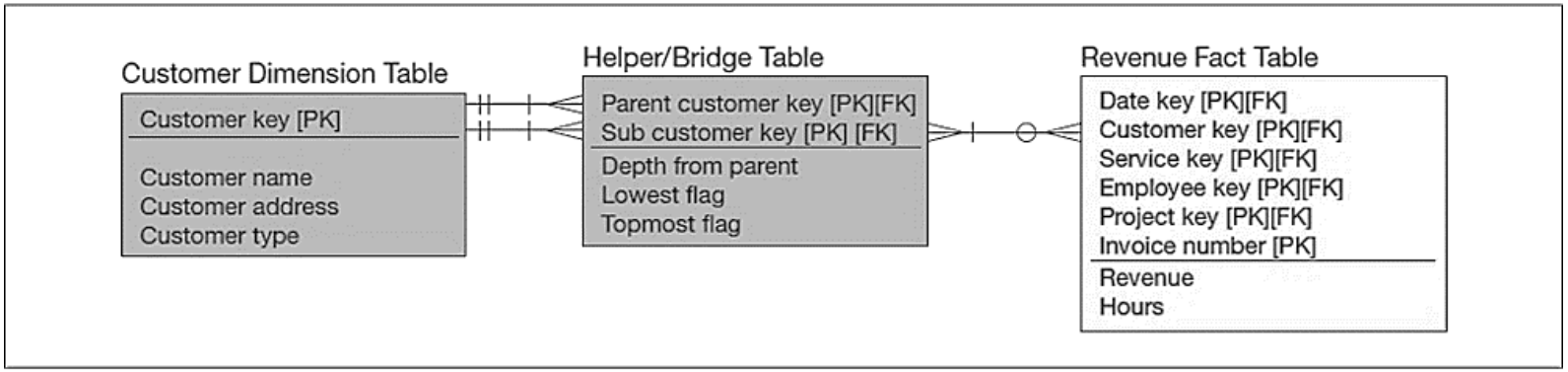
Example: Normalizing Dimensional Table Using Hierarchy

- Dimension hierarchies help to provide levels of aggregation for users wanting summary information in a data warehouse.

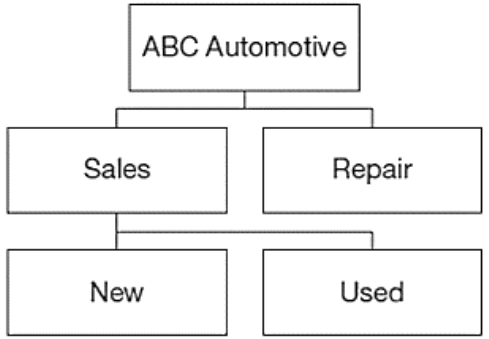


Example: Representing Hierarchical Relationships within a Dimension

(a) Use of a helper table



(b) Sample hierarchy with customer and helper tables



Customer Table

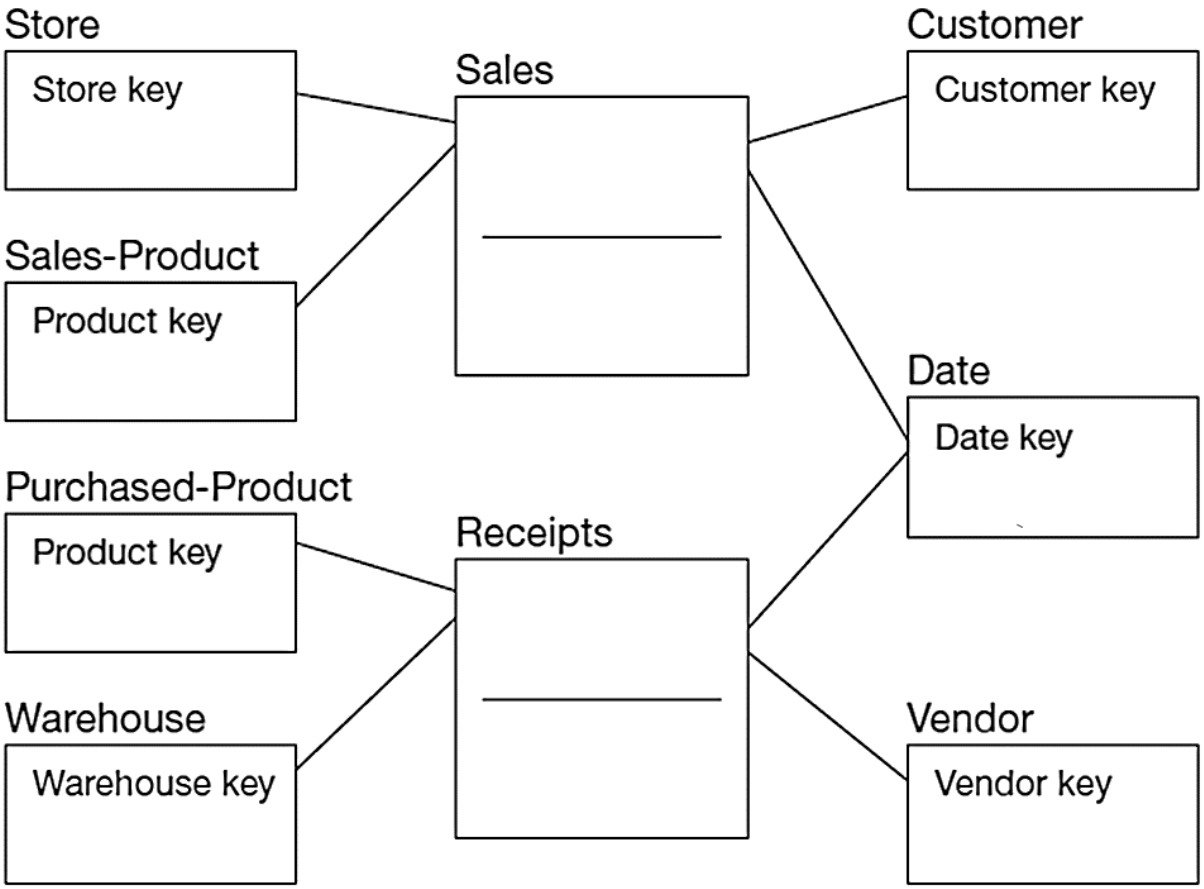
Customer key	Name	Address	Type
C0000001	ABC Automotive	100 1st St.	Dealer
C0000002	ABC Auto Sales	110 1st St.	Sales
C0000003	ABC Repair	130 1st St.	Service
C0000004	ABC Auto New Sales	110 1st St.	Sales
C0000005	ABC Auto Used Sales	110 1st St.	Sales

Hierarchy Helper Table

Parent key	Sub key	Depth	Lowest	Topmost
C0000001	C0000001	0	N	Y
C0000001	C0000002	1	N	N
C0000001	C0000003	1	N	N
C0000001	C0000004	2	Y	N
C0000001	C0000005	2	Y	N
C0000002	C0000002	0	N	N
C0000002	C0000004	1	Y	N
C0000002	C0000005	1	Y	N
C0000003	C0000003	0	Y	N
C0000004	C0000004	0	Y	N
C0000005	C0000005	0	Y	N

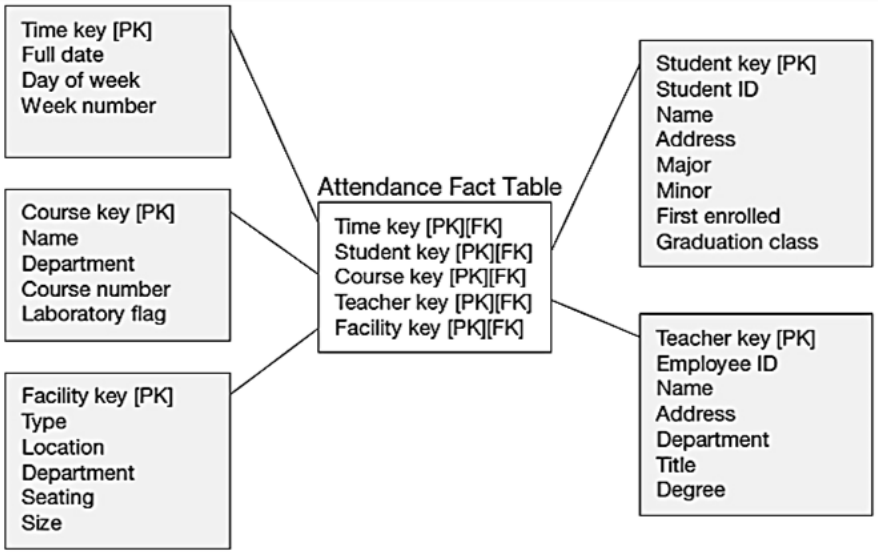
Multiple Fact Tables

- Two fact tables connect two star schemas. Conformed dimension is associated with multiple fact tables.



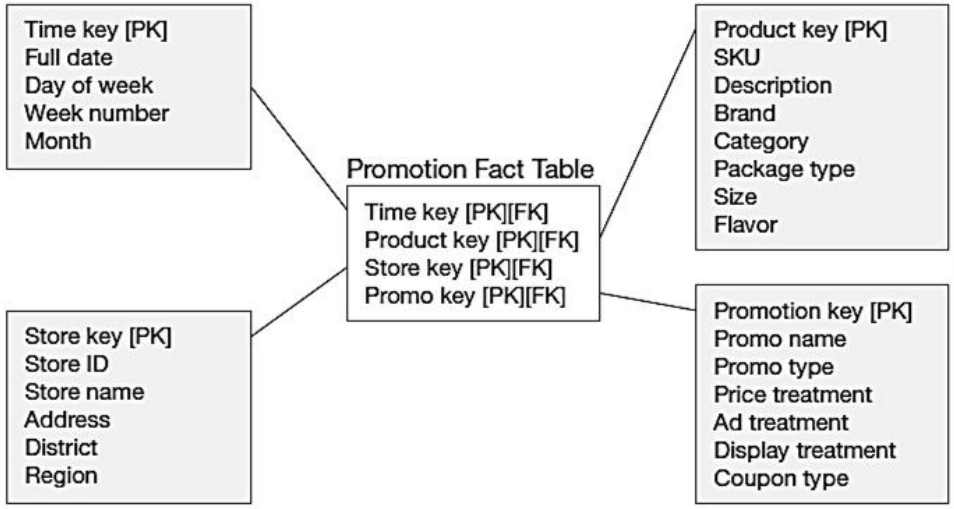
Factless Fact Table

- No data in fact table, just keys associating dimension records. Fact table forms an n-ary relationship between dimensions



Factless fact table showing occurrence of an event

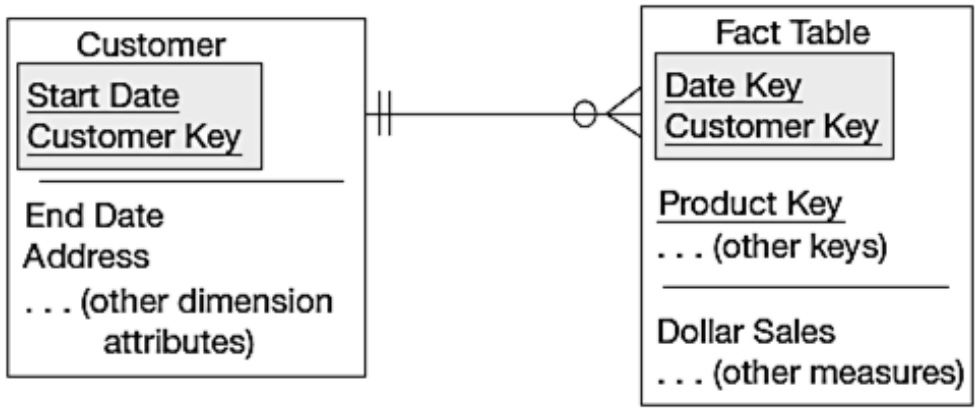
Factless fact table showing coverage



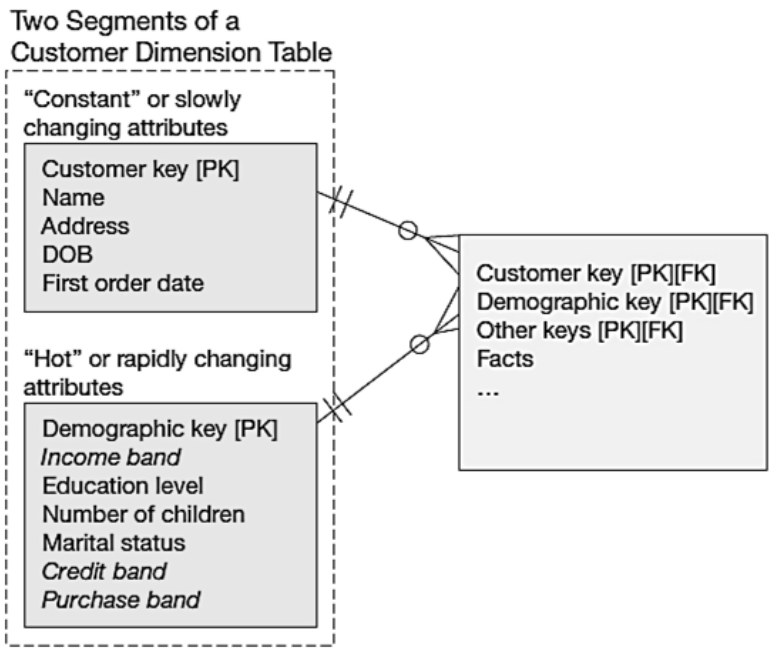
Slowly Changing Dimensions (*optional*)

How to maintain knowledge of the past?

- Kimball's approaches:
  - Type 1: just replace old data with new (lose historical data)
  - Type 2: for each changing attribute, create a current value field and several old-valued fields (multivalued)
  - Type 3: create a new dimension table row each time the dimension object changes, with all dimension characteristics at the time of change — most common approach



(a) Example of Type 3



(b) Dimension Segmentation

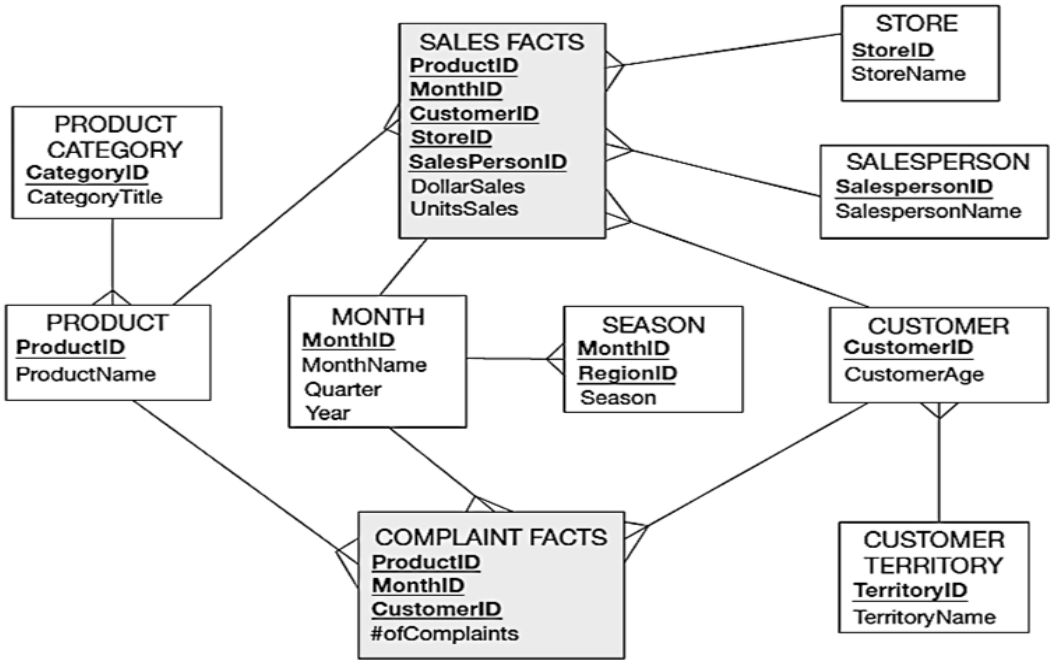
Determining Dimensions and Facts

Department of MANAGEMENT Clemson University

1. What was the dollar sales of health and beauty products in North America to customers over the age of 50 in each of the past three years? 2. What is the name of the salesperson who had the highest dollar sales of each product in the first quarter of this year? 3. How many European customer complaints did we receive on pet food products during the past year? How has it changed from month to month this year? 4. What is the name of the store(s) that had the highest average monthly quantity sales of casual clothing during the summer?	dollar sales	number of complaints	avg. qty. sales
product category	1	3	4
customer territory	1	3	
customer age	1		
year	1	3	
salesperson name	2		
product	2		
quarter	2		
month		3	
store			4
season			4

(a) fact-qualifier matrix for sales and customer service tracking

(b) Star schema for sales and customer service tracking



10 Essential Rules for Dimensional Modeling

Department of MANAGEMENT Clemson University

1. **Use atomic facts:** Eventually, users want detailed data, even if their initial requests are for summarized facts.
2. **Create single-process fact tables:** Each fact table should address the important measurements for one business process, such as taking a customer order or placing a material purchase order.
3. **Include a date dimension for every fact table:** A fact should be described by the characteristics of the associated day (or finer) date/time to which that fact is related.
4. **Enforce consistent grain:** Each measurement in a fact table must be atomic for the same combination of keys (the same grain).
5. **Disallow null keys in fact tables:** Facts apply to the combination of key values, and helper tables may be needed to represent some M:N relationships.
6. **Honor hierarchies:** Understand the hierarchies of dimensions and carefully choose to snowflake the hierarchy or denormalize into one dimension.
7. **Decode dimension tables:** Store descriptions of surrogate keys and codes used in fact tables in associated dimension tables, which can then be used to report labels and query filters.
8. **Use surrogate keys:** All dimension table rows should be identified by a surrogate key, with descriptive columns showing the associated production and source system keys.
9. **Conform dimensions:** Conformed dimensions should be used across multiple fact tables.
10. **Balance requirements with actual data:** Unfortunately, source data may not precisely support all business requirements, so you must balance what is technically possible with what users want and need.

Source: Based on Ross (2009).

The Process of Data Analysis

Department of MANAGEMENT Clemson University

- Accessing data sources
- Combining tables
- Transforming variables and cleaning data
- Exploring and describing data
- Visualizing patterns in the data
- Analyzing and modeling data

Clemson *Means*
BUSINESS

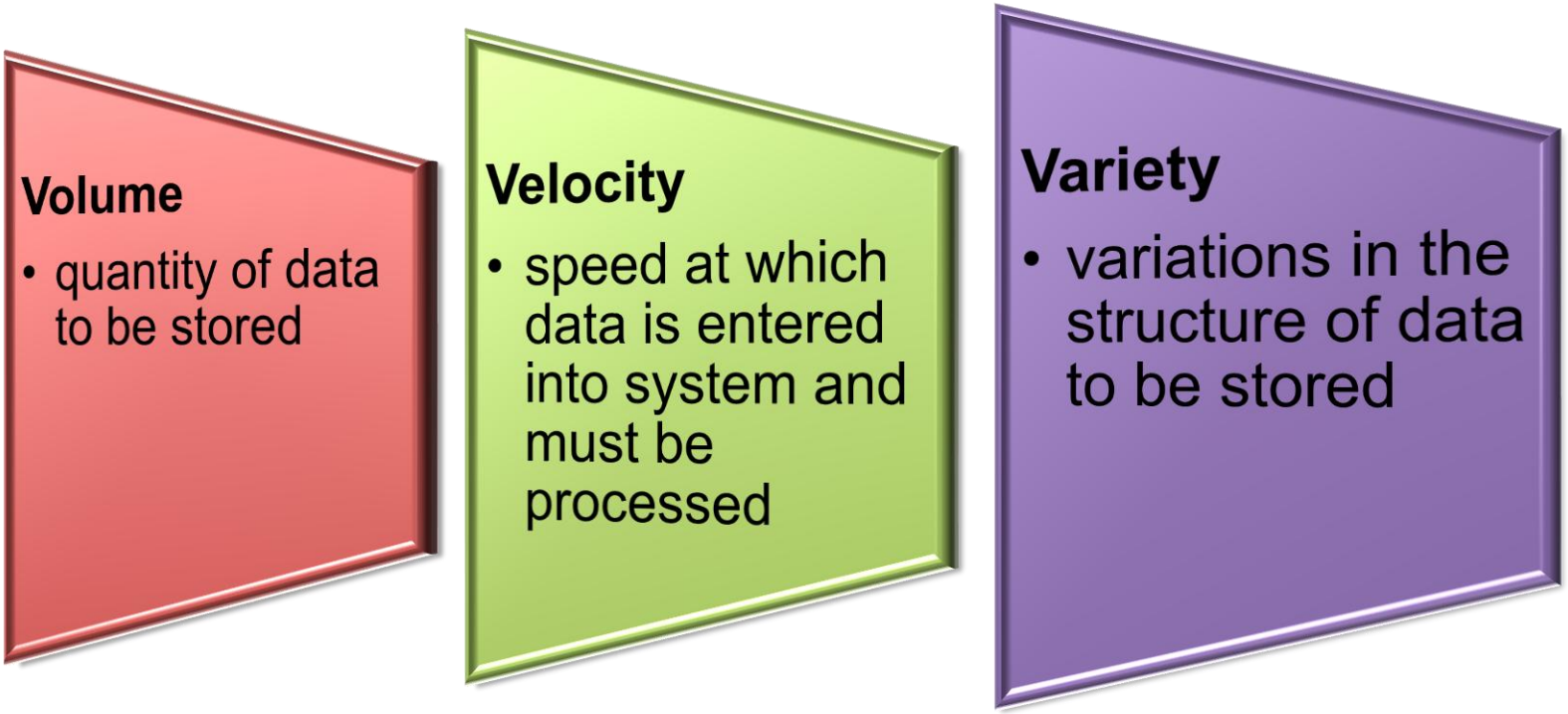
Chapter 6
Big Data

Instructor: He Li

Big Data

Department of MANAGEMENT Clemson University

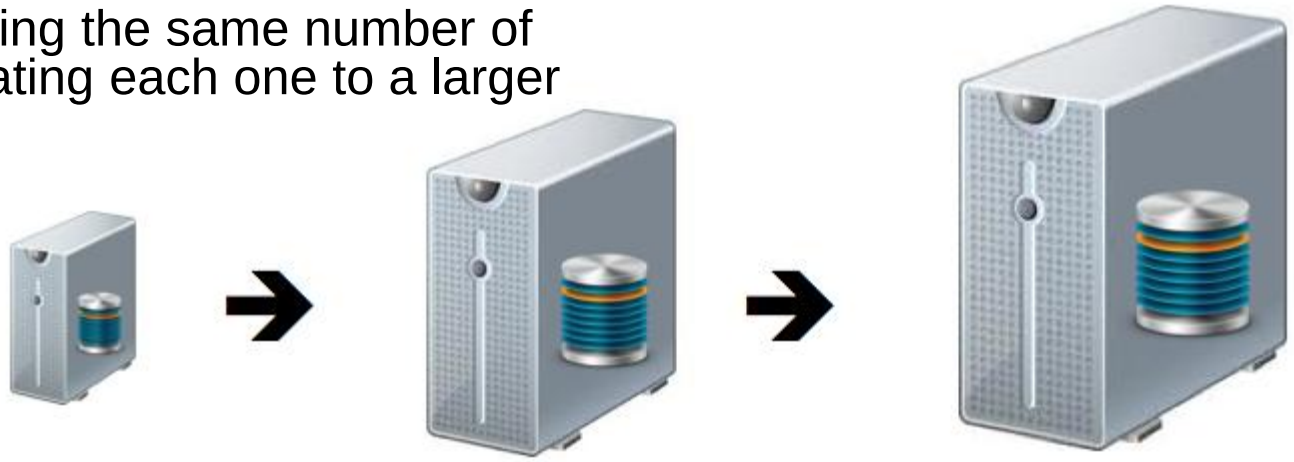
Definition: Data that exist in very large volumes and many different varieties (data types) that need to be processed at a very high velocity (speed).



Big Data – Volume (extremely large storage capacities)

Department of MANAGEMENT Clemson University

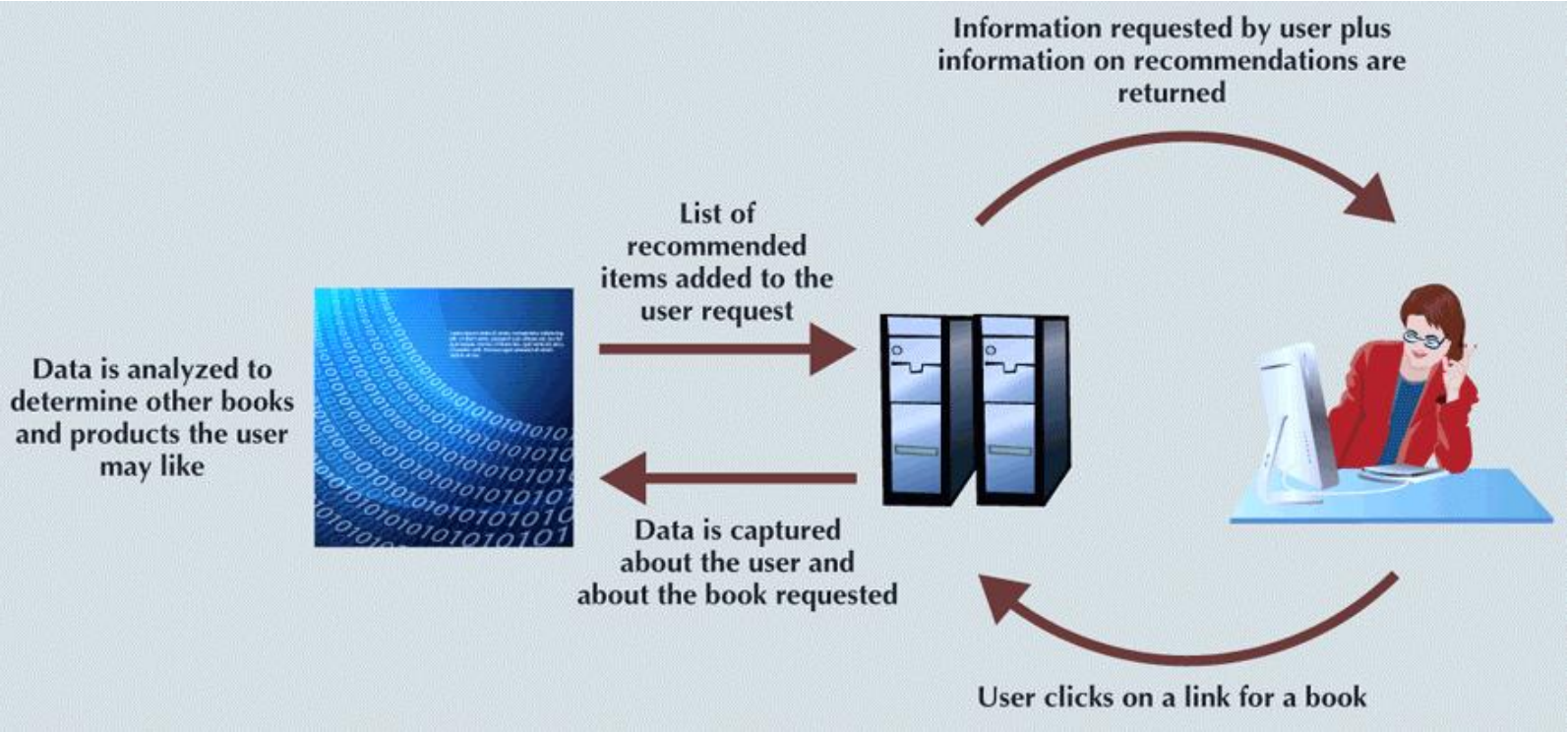
Scaling up: keeping the same number of systems but migrating each one to a larger system



Scaling out: when the workload exceeds server capacity, it is spread out across many servers

Big Data – Velocity (speed at which data is entered into system and must be processed)

- **Stream processing:** focuses on input processing and requires analysis of data stream as it enters the system
- **Feedback loop processing:** analysis of data to produce actionable results



Big Data – Variety

Department of MANAGEMENT Clemson University

Unstructured data:
does not fit into a predefined model (e.g., maps, satellite images, emails, texts, tweets, and videos)

The university has 5600 students.
John's ID is number 1, he is 18 years old and already holds a B.Sc. degree.
David's ID is number 2, he is 31 years old and holds a Ph.D. degree. Robert's ID is number 3, he is 51 years old and also holds the same degree as David, a Ph.D. degree.

Semistructured data:
some parts of the data fit a predefined model while other parts do not

```
<University>
 <Student ID="1">
  <Name>John</Name>
  <Age>18</Age>
  <Degree>B.Sc.</Degree>
 </Student>
 <Student ID="2">
  <Name>David</Name>
  <Age>31</Age>
  <Degree>Ph.D. </Degree>
 </Student>
....
</University>
```

Structured data:
fits into a predefined data model

ID	Name	Age	Degree
1	John	18	B.Sc.
2	David	31	Ph.D.
3	Robert	51	Ph.D.
4	Rick	26	M.Sc.
5	Michael	19	B.Sc.

Other Characteristics of Big Data

Department of MANAGEMENT Clemson University

- **Variability:** changes in meaning of data based on context
- **Veracity:** trustworthiness of data
- **Value:** degree data can be analyzed for meaningful insight
- **Visualization:** ability to graphically resent data to make it understandable

- Relational databases are not necessarily the best for storing and managing all organizational data
  - Polyglot persistence: coexistence of a variety of data storage and management technologies within an organization's infrastructure

NoSQL (*Not Only SQL*)

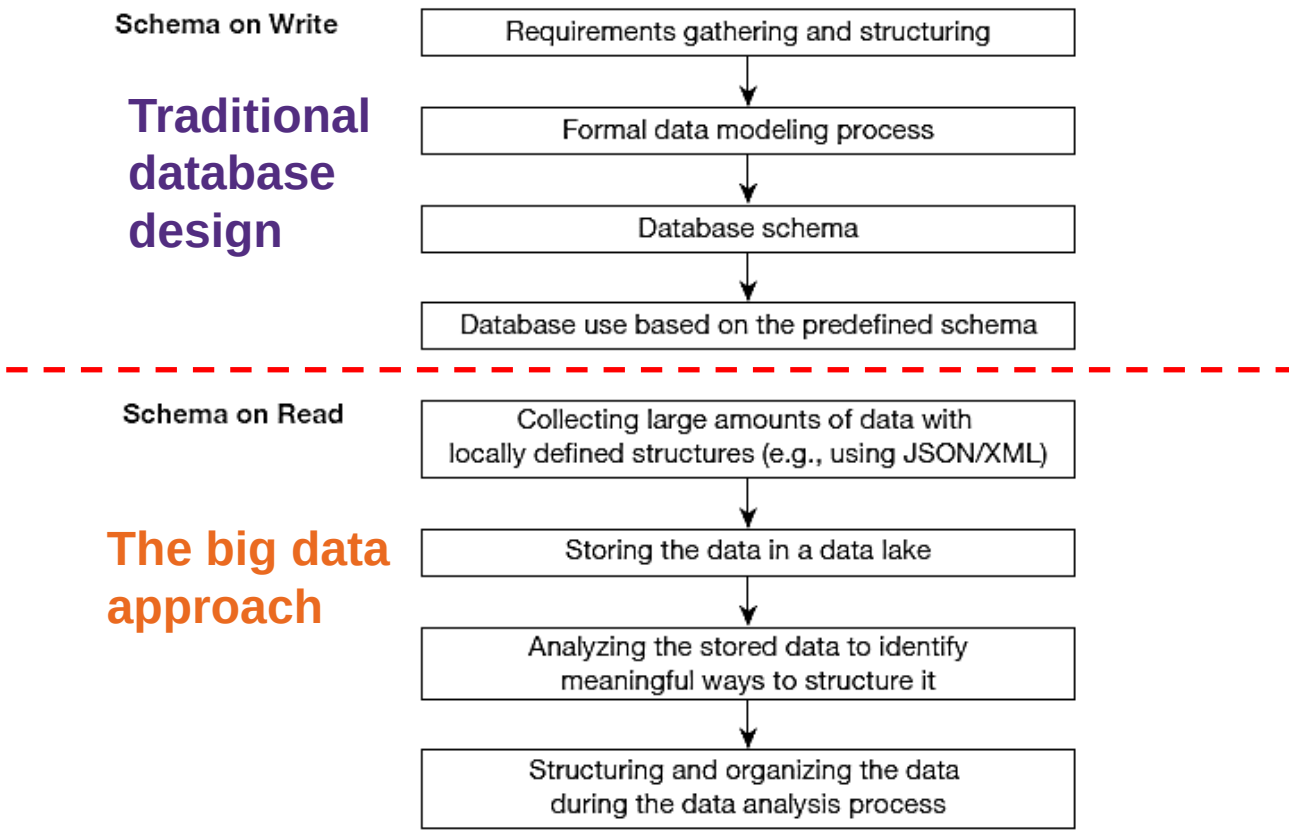
Department of MANAGEMENT Clemson University

- Scaling out rather than scaling up
- Natural for a cloud environment
- Largely open source

NoSQL Category	Example Databases	Developer
Key-Value Databases	Redis Dynamo	Redis Labs Amazon
Document Databases	MongoDB CouchDB	MongDB, Inc Apache
Columnar Databases	Hbase Cassandra	Apache Apache (Originally Facebook)
Graph Databases	Neo4J GraphBase	Neo4j FactNexus

NoSQL

- Supports schema on read
- Not ACID compliant!
- BASE – basically available, soft state, eventually consistent



NoSQL Classifications: Key-Value Databases

- A simple pair of a key and an associated collection of values. Key is usually a string. Database has no knowledge of the structure or meaning of the values.
- Example: Redis

Bucket = Customer

Key	Value
10010	"LName Ramas FName Alfred Initial A Areacode 615 Phone 844-2573 Balance 0"
10011	"LName Dunne FName Leona Initial K Areacode 713 Phone 894-1238 Balance 0"
10014	"LName Orlando FName Myron Areacode 615 Phone 222-1672 Balance 0"

NoSQL Classifications: Document Databases

Department of MANAGEMENT Clemson University

- Like a key-value store, but “document” goes further than “value”. Document is structured so specific elements can be manipulated separately.
- Example: Mongo DB

Collection = Customer

Key	Document
10010	{LName: “Ramas”, FName: “Alfred”, Initial: “A”, Areacode: “615”, Phone: “844-2573”, Balance: “0”}
10011	{LName: “Dunne”, FName: “Leona”, Initial: “K”, Areacode: “713”, Phone: “894-1238”, Balance: “0”}
10014	{LName: “Orlando”, FName: “Myron”, Areacode: “615”, Phone: “222-1672”, Balance: “0”}

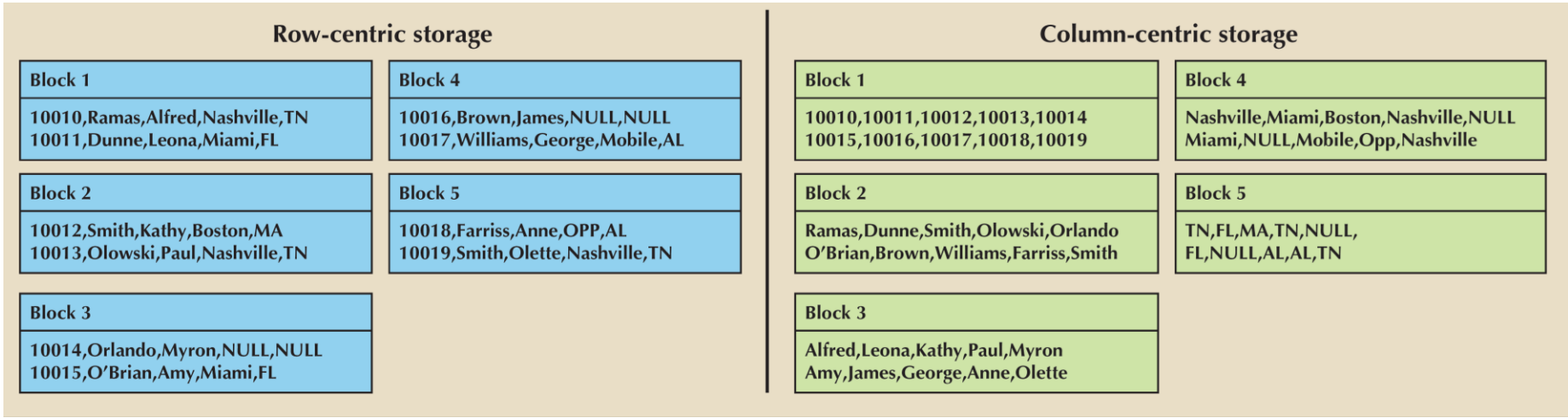
NoSQL Classifications: Columnar Databases

- Rows and columns. Distribution of data based on both key values (records or rows) and columns, using "column groups/families"

- Example: Apache Cassandra

CUSTOMER relational table

Cus_Code	Cus_LName	Cus_FName	Cus_City	Cus_State
10010	Ramas	Alfred	Nashville	TN
10011	Dunne	Leona	Miami	FL
10012	Smith	Kathy	Boston	MA
10013	Olowski	Paul	Nashville	TN
10014	Orlando	Myron		
10015	O'Brian	Amy	Miami	FL
10016	Brown	James		
10017	Williams	George	Mobile	AL
10018	Farriss	Anne	Opp	AL
10019	Smith	Olette	Nashville	TN



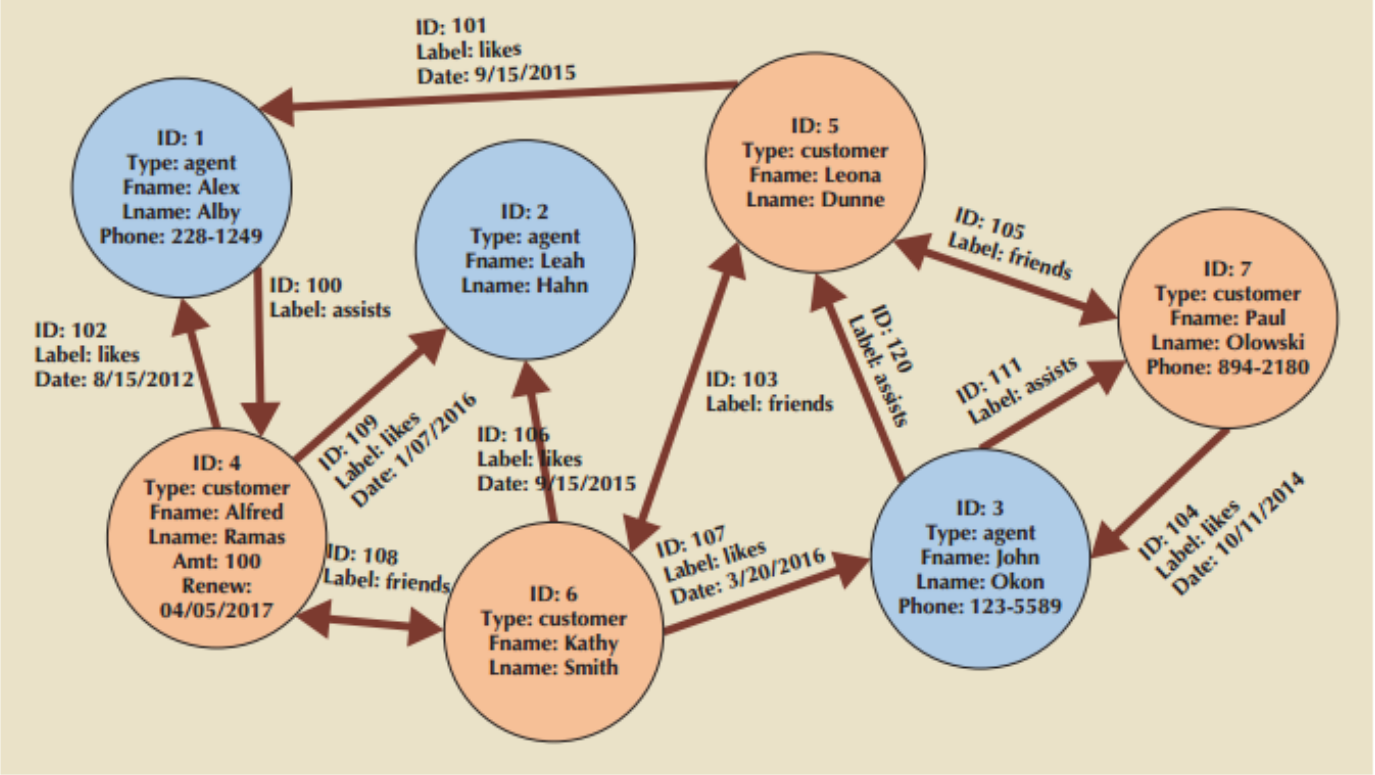
NoSQL Classifications: Columnar Databases

Department of MANAGEMENT Clemson University

Column Family Name	CUSTOMERS
Key	Rowkey 1
Columns	City: Nashville Fname: Alfred Lname: Ramas State: TN
Key	Rowkey 2
Columns	Balance: 345.86 Fname: Kathy Lname: Smith
Key	Rowkey 3
Columns	Company: Local Markets, Inc. Lname: Dunne

NoSQL Classifications: Graph Databases

- Maintain information regarding the relationships between data items. Nodes with properties. Connections between nodes (relationships) can also have properties.
- Example: Neo4j

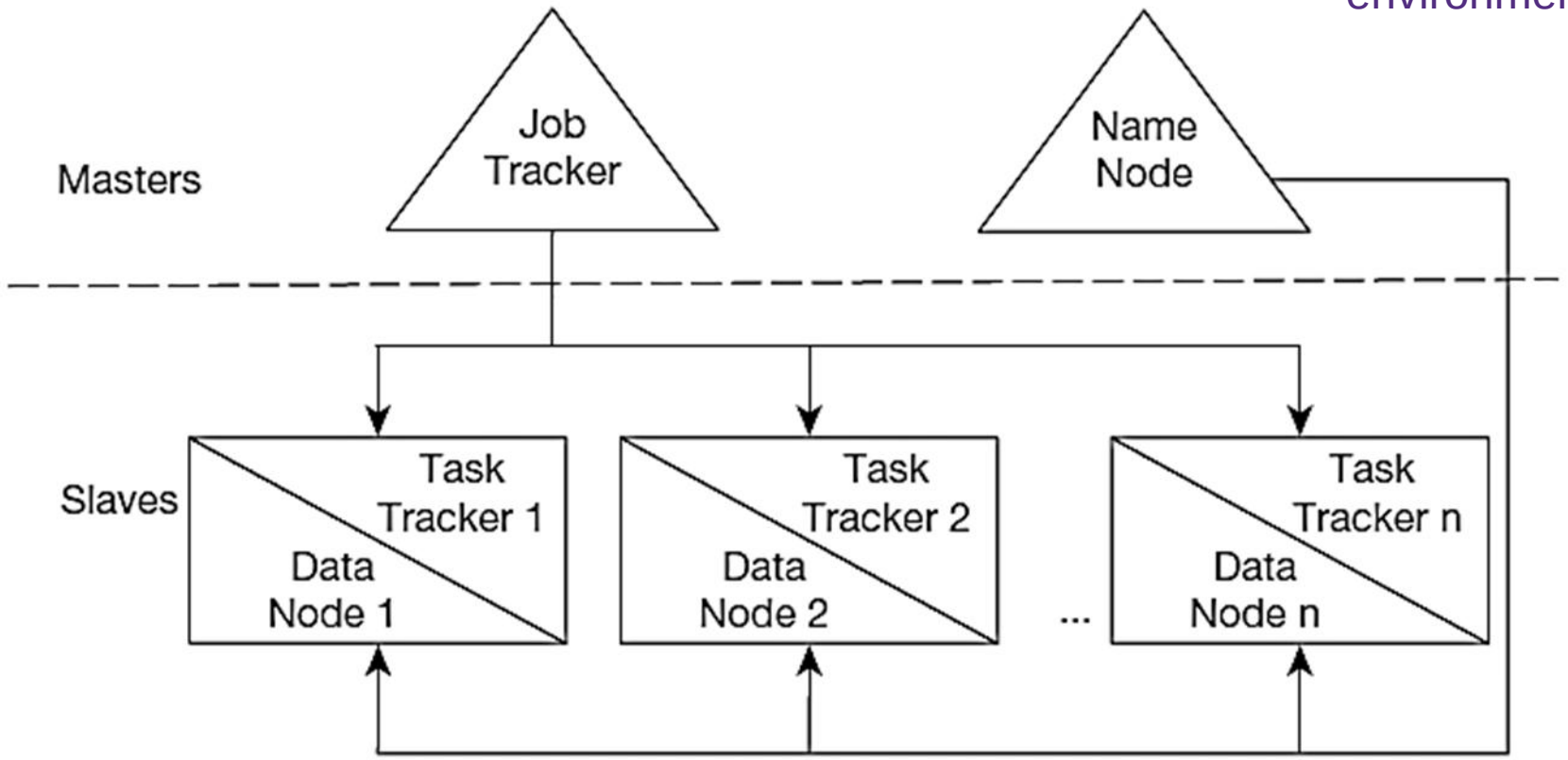


- an open source implementation framework of MapReduce
- the most talked about Big-Data data management product today

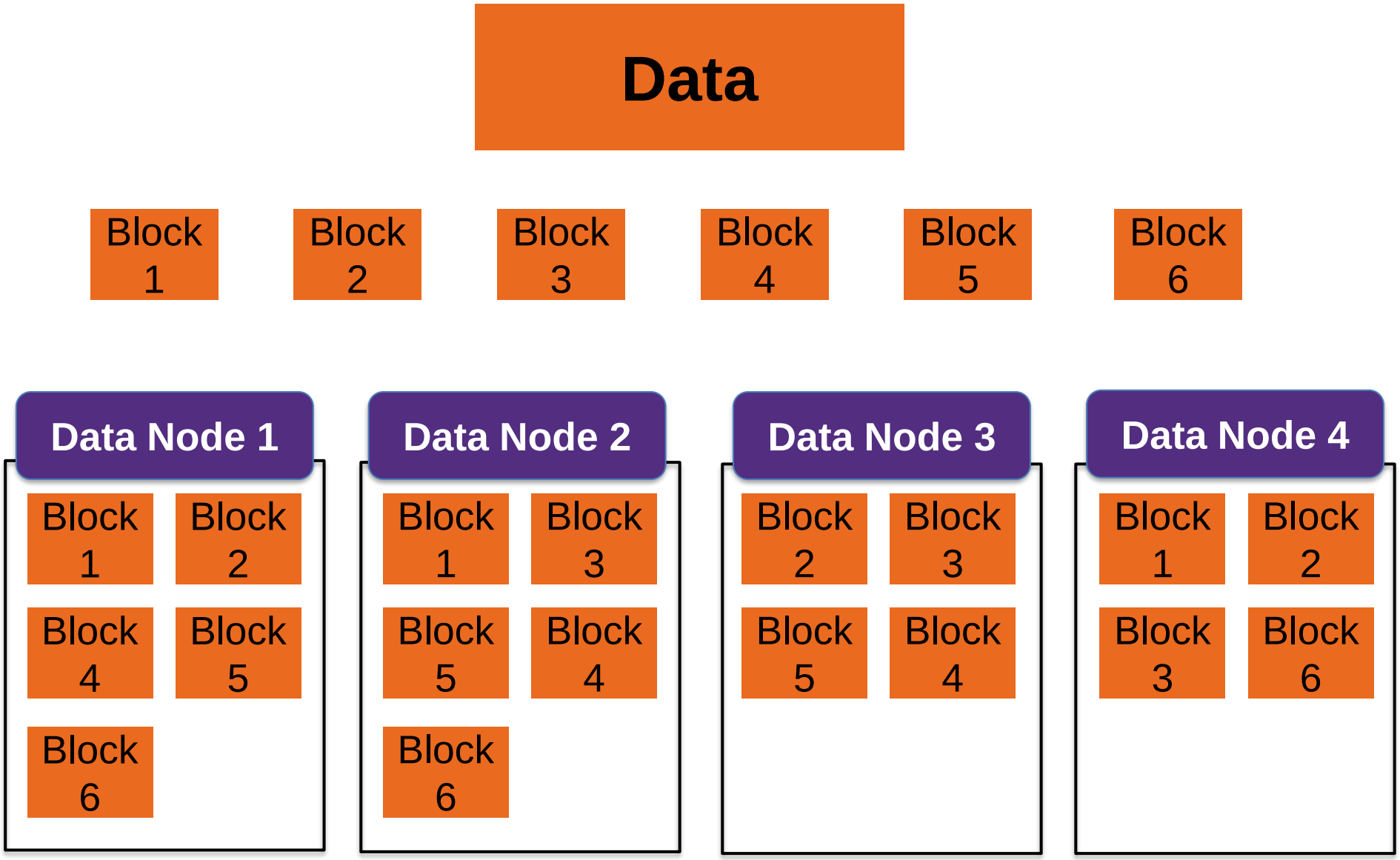
- widely adopted in companies:

MapReduce: an algorithm for massive parallel processing of various types of computing tasks

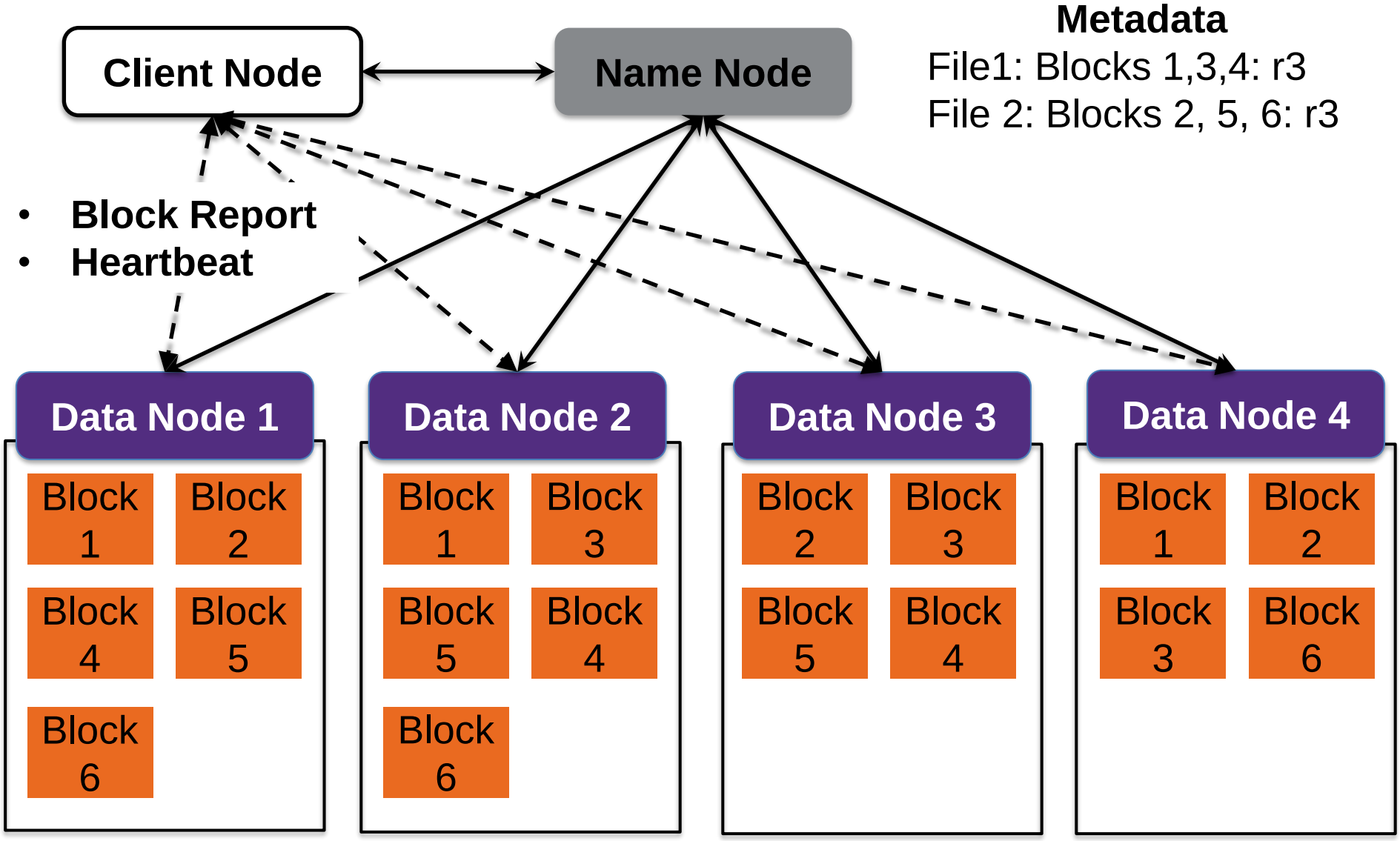
HDFS: a file system designed for managing many large files in a distributed environment



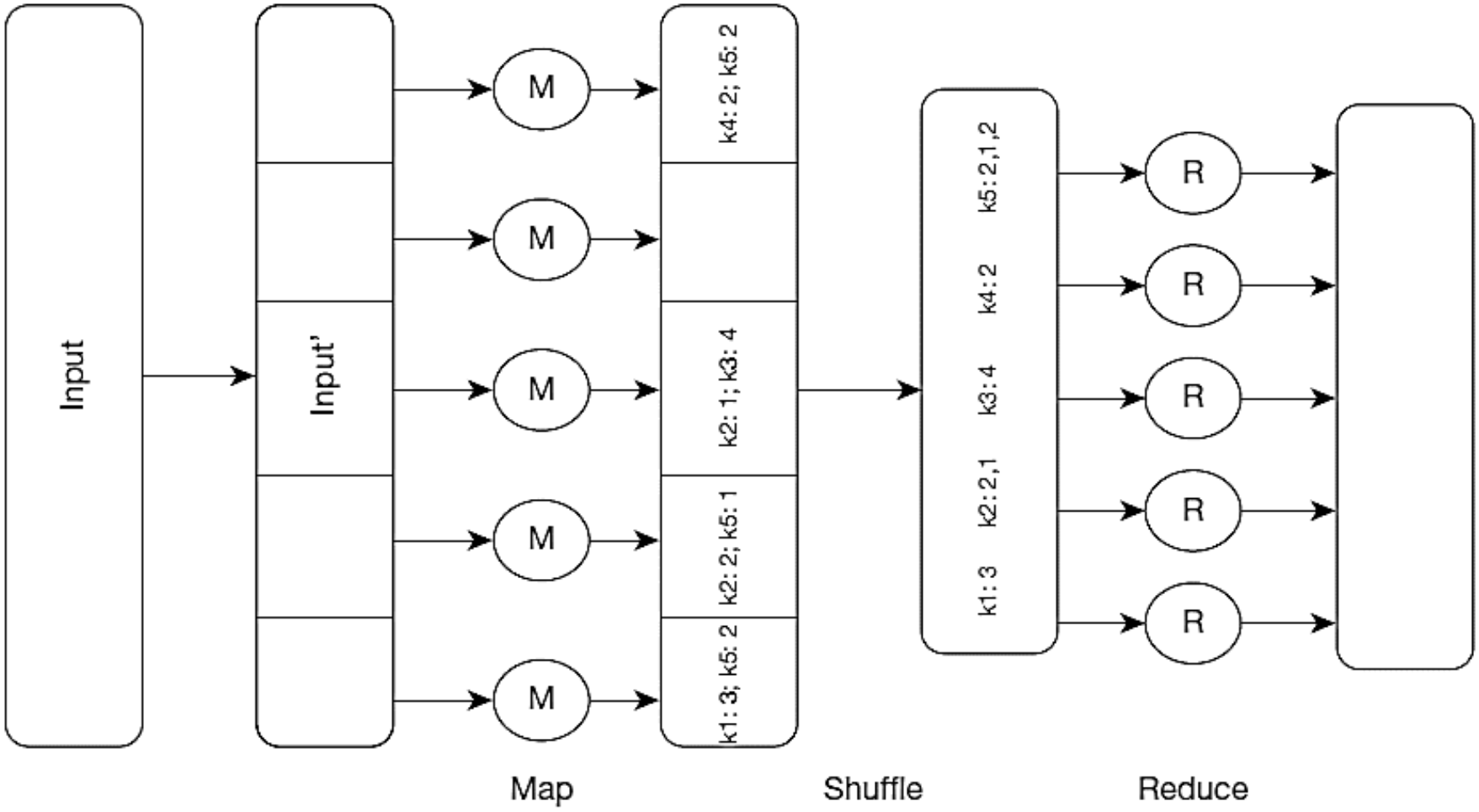
How Does HDFS Work?



How Does HDFS Work?



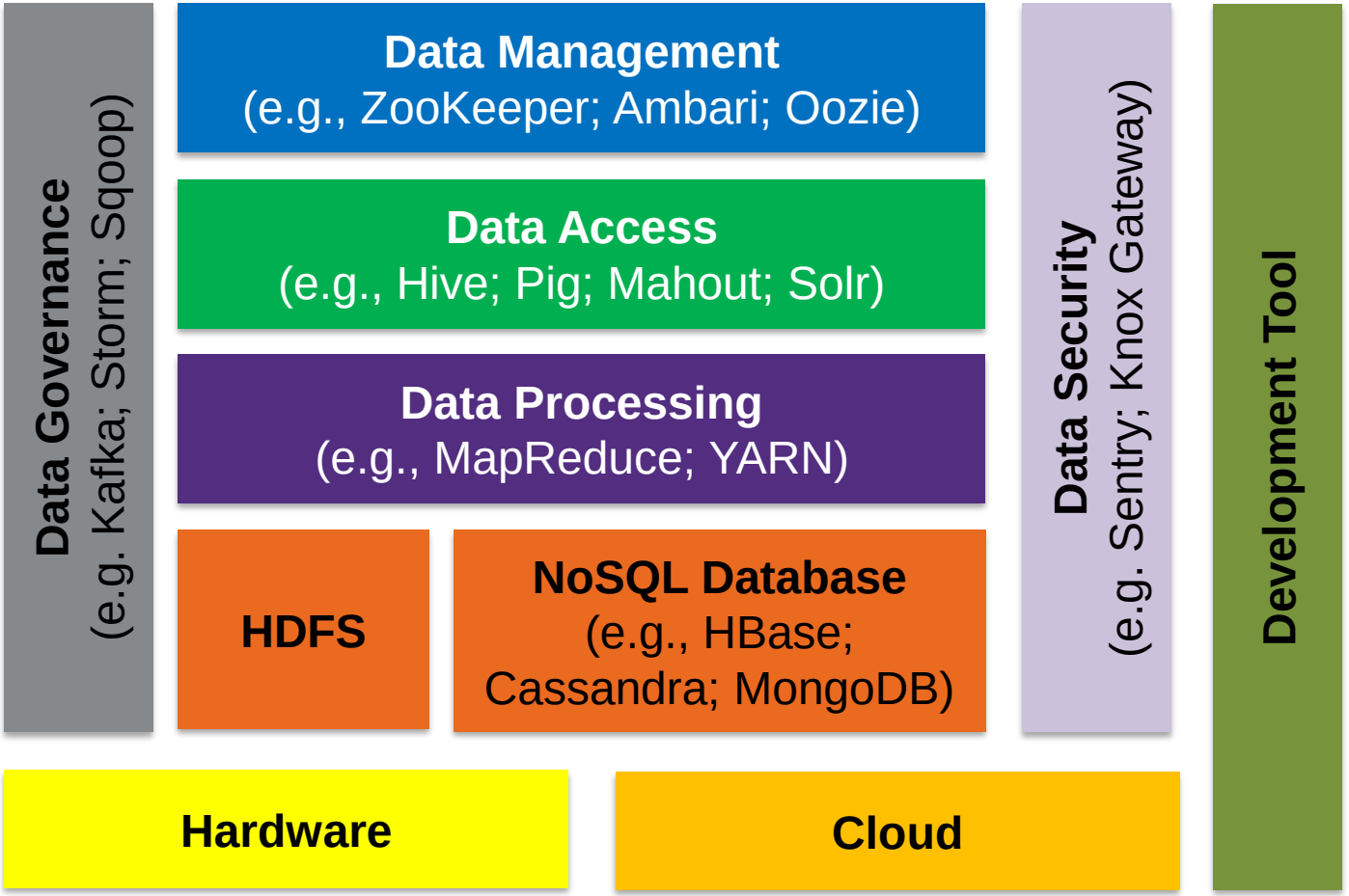
MapReduce



MapReduce: Simplified Data Processing on Large Clusters, Jeff Dean, Sanjay Ghemawat, Google, I n c., https://research.google.com/archive/mapreduce-osdi04-slides/index-auto-0007.html. Courtesy of the authors.

Hadoop Ecosystem

Department of MANAGEMENT Clemson University



Hadoop Vendors

Department of MANAGEMENT Clemson University